

claude-windows / 96963dab-d38f-488c-af44-530ddbfeed1

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\96963dab-d38f-488c-af44-530ddbfeed1\subagents\workflows\wf_00bebd3c-82f\agent-a0287d595cecb5f3e.jsonl`  
- SHA-256: `131b4d1163251b0dfaecd52311732ae78a81fb4c20bf142de8b3cfda18266e7e`  
- Source modified: `2026-07-08T05:51:19+00:00`  
- Imported at: `2026-07-08T15:59:27+00:00`  
- project: `wf_00bebd3c-82f`  
- session_id: `96963dab-d38f-488c-af44-530ddbfeed1`
```

Transcript

1. user (2026-07-08T05:44:49.640Z)

You are mapping a subsystem of the repo at C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer (Windows checkout; read-only ? do NOT edit anything).
Context: GitHub issue #524 wants to (a) move validation onto the L4 Cloud Run Job worker via a config-driven phase_placement knob, (b) drop a redundant re-hash in /api/process by trusting the upload's streamed video_id, and (c) design a topology where upload->stitch runs on the L4 with async copy-to-bucket. You are producing the precise code map a designer/implementer needs. Cite exact file paths and line numbers. Read the actual code, not just docs. Follow imports where the flow crosses files.
Subsystem: VALIDATION. Map the validators subsystem: registry, which validators run (scene_cut/blur/relevance?), the #512 shared-decode single-pass, the #518 validation-verdict cache keyed by video_id (where stored ? DB table? column?), how a rejected video surfaces to the user/job, and validation's inputs (does it need the WHOLE local file, or could it stream/partial-read?). Also: what does validation produce (verdict shape) and where is it persisted? Note exactly what a worker-side validation run would need to produce for the CP to record an identical verdict, and how the #518 cache interacts with moving validation off the CP.

2. user (2026-07-08T05:44:58.076Z)

```
backend\pipeline\validators\__init__.py  
backend\pipeline\validators\format.py  
backend\pipeline\validators\pts_continuity.py  
backend\pipeline\validators\resolution_fps.py  
backend\pipeline\validators\blur.py  
backend\pipeline\validators\frame_sampler.py  
backend\pipeline\validators\relevance.py  
backend\pipeline\validators\scene_cut.py  
backend\pipeline\validators\__pycache__\__init__.cpython-313.pyc  
backend\pipeline\validators\__pycache__\base.cpython-313.pyc  
backend\pipeline\validators\__pycache__\blur.cpython-313.pyc  
backend\pipeline\validators\__pycache__\format.cpython-313.pyc  
backend\pipeline\validators\__pycache__\frame_sampler.cpython-313.pyc  
backend\pipeline\validators\__pycache__\pts_continuity.cpython-313.pyc  
backend\pipeline\validators\__pycache__\relevance.cpython-313.pyc  
backend\pipeline\validators\__pycache__\resolution_fps.cpython-313.pyc  
backend\pipeline\validators\__pycache__\scene_cut.cpython-313.pyc  
backend\pipeline\validators\base.py
```

3. user (2026-07-08T05:45:00.233Z)

Found 30 files
backend\pipeline\validators\base.py
backend\orchestration.py
backend\infrastructure\database_media.py
backend\infrastructure\database.py

```
backend\ui\assets\index-kJhH4dVE.js
backend\worker\__main__.py
backend\compute\base.py
backend\compute\cloud_run.py
backend\pipeline\validators\blur.py
backend\pipeline\validators\frame_sampler.py
backend\pipeline\validators\relevance.py
backend\pipeline\validators\scene_cut.py
backend\config\models.py
backend\pipeline\segmenters\trajectory_hybrid.py
backend\pipeline\segmenters\gemini.py
backend\main.py
backend\infrastructure\database_jobs.py
backend\api\job_worker.py
backend\api\uploads.py
backend\pipeline\validators\resolution_fps.py
backend\pipeline\validators\pts_continuity.py
backend\reporting\harvest.py
backend\pipeline\segmenters\base.py
backend\eval\tolerance_metrics.py
backend\eval\calibration.py
backend\sports\base.py
backend\api\static\app.js
backend\reporting\spec.yaml
backend\api\static\index.html
backend\api\static\style.css
```

4. user (2026-07-08T05:45:08.413Z)

```
1     import hashlib
2     import json
3     import logging
4     from abc import ABC, abstractmethod
5     from typing import Any, List
6
7     from pydantic import BaseModel
8
9     from backend.pipeline.validators.frame_sampler import (
10         FRAME_META_KEY,
11         FRAME_STATES_KEY,
12         SampleMeta,
13         run_frame_sampling,
14     )
15
16     logger = logging.getLogger(__name__)
17
18     # Bumped when validator LOGIC (not config) changes the verdict for identical
19     bytes+config ? e.g. a
20     # validator is rewritten, or the sampling arithmetic changes. It rides inside
21     config_fingerprint, so
22     # a bump invalidates every cached validation verdict (see backend/orchestration.py's
23     validation cache
24     # + backend/infrastructure/database.py v10). The validator SET and each validator's
25     config-derived
26     # thresholds are already folded into the fingerprint; this covers changes the config
27     can't express.
28     _VALIDATION_FINGERPRINT_SCHEMA = 1
29
30
31     class ValidationResult(BaseModel):
32         validator_name: str
33         passed: bool
34         message: str
35         details: dict[str, Any] = {}
```

```

32
33 class VideoValidator(ABC):
34     @property
35     @abstractmethod
36     def name(self) -> str:
37         """Unique identifier for this validator."""
38         pass
39
40     @property
41     @abstractmethod
42     def description(self) -> str:
43         """Human-readable explanation of what this validator checks."""
44         pass
45
46     @abstractmethod
47     def validate(
48         self, video_path: str, config: Any, context: dict[str, Any]
49     ) -> ValidationResult:
50         """
51         Executes validation.
52         - video_path: Local path to the MP4 file.
53         - config: AppConfig (typed) ? callers pass the typed model; attribute access
54           preferred over .get() dict chains.
55         - context: Dynamic context dict shared across validators (stores shared values
56           like ffprobe metadata).
57         """
58         pass
59
60
61 class FrameSamplingValidator(VideoValidator):
62     """A validator whose work is sampling decoded frames.
63
64     The three frame-decoding validators (scene_cut, blur, relevance) used to each
65     open their own ``cv2.VideoCapture`` and seek to their sample frames ? ~125
66     random ``POS_FRAMES`` seeks across three decode opens, which cost minutes on
67     long-GOP 4K clips (see ``frame_sampler``). Subclasses instead declare *which*
68     frame indices they want (:meth:`sample_indices`) and reduce those frames
69     online (:meth:`_init_state` / :meth:`_consume`), so the registry can decode
70     the whole set *once* in a single shared forward pass.
71
72     :meth:`validate` uses the shared pass' result when the registry ran it; when
73     a subclass is invoked standalone (direct ``.validate`` call, no shared
74     context) it decodes its own indices in one pass via the same engine. Either
75     way the frames ? and therefore the pass/fail decision ? are identical to the
76     old per-validator seeks.
77     """
78
79     @abstractmethod
80     def sample_indices(self, total_frames: int, fps: float) -> List[int]:
81         """Absolute frame indices this validator samples, in ascending order.
82
83         Pure and deterministic in ``(total_frames, fps)`` ? the shared pass unions
84         these across validators to drive one decode, and the regression suite pins
85         them to the legacy per-validator seek arithmetic.
86         """
87         ...
88
89     @abstractmethod
90     def _init_state(self, config: Any) -> Any:
91         """Fresh accumulator for one run (may pre-resolve config-derived params)."""
92         ...
93
94     @abstractmethod
95     def _consume(self, state: Any, frame_idx: int, frame: Any) -> None:
96         """Fold one sampled frame (BGR ndarray) into ``state``. Called in index

```

```

order."""
97         ...
98
99     @abstractmethod
100     def _finalize(
101         self, state: Any, meta: SampleMeta, config: Any, context: dict[str, Any]
102     ) -> ValidationResult:
103         """Render the ValidationResult from the accumulated ``state`` + decode meta."""
104         ...
105
106     def validate(
107         self, video_path: str, config: Any, context: dict[str, Any]
108     ) -> ValidationResult:
109         states = context.get(FRAME_STATES_KEY)
110         if states is not None and self.name in states:
111             # Shared single pass already decoded and reduced our frames.
112             meta = context[FRAME_META_KEY]
113             state = states[self.name]
114         else:
115             # Standalone: decode just this validator's indices in one pass.
116             meta, own_states = run_frame_sampling(video_path, [self], config)
117             state = own_states[self.name]
118         return self._finalize(state, meta, config, context)
119
120
121 class ValidationRegistry:
122     def __init__(self):
123         self._validators: List[VideoValidator] = []
124
125     def register(self, validator: VideoValidator):
126         """Registers a new validator."""
127         self._validators.append(validator)
128
129     def run_all_with_context(self, video_path: str, config: Any):
130         """Runs all validators and returns ``(results, context)``.
131
132         The per-run ``context`` accumulates shared values (notably
133         ``context['ffprobe_meta']`` produced by FormatValidator). ``run_all``
134         discards it; callers that want media metadata for an observability report
135         use this variant instead of re-probing.
136         """
137         results = []
138         context: dict[
139             str, Any
140         ] = {} # Shared context (e.g. for caching ffprobe outputs)
141         self._prepare_frame_samples(video_path, config, context)
142         for val in self._validators:
143             try:
144                 res = val.validate(video_path, config, context)
145                 results.append(res)
146             except Exception as e:
147                 results.append(
148                     ValidationResult(
149                         validator_name=val.name,
150                         passed=False,
151                         message=f"Validator threw an unexpected error: {str(e)}",
152                     )
153                 )
154         return results, context
155
156     def _prepare_frame_samples(
157         self, video_path: str, config: Any, context: dict[str, Any]
158     ) -> None:
159         """Decode every frame-sampling validator's frames in ONE shared pass.
160

```

```

161     Populates ``context`` with the shared decode meta + per-validator
162     accumulator state; each :class:`FrameSamplingValidator` then finalizes
163     from that instead of opening its own capture. Best-effort: any failure
164     here is swallowed so validators transparently fall back to a standalone
165     pass (the run must never crash on a decode hiccup)."""
166     samplers = [
167         v for v in self._validators if isinstance(v, FrameSamplingValidator)
168     ]
169     if not samplers:
170         return
171     try:
172         meta, states = run_frame_sampling(video_path, samplers, config)
173     except Exception:
174         logger.exception(
175             "Shared frame-sampling pass failed for %s; validators will fall "
176             "back to standalone decode.",
177             video_path,
178         )
179     return
180     context[FRAME_META_KEY] = meta
181     context[FRAME_STATES_KEY] = states
182
183     def run_all(self, video_path: str, config: Any) -> List[ValidationResult]:
184         """Runs all registered validators in sequence, accumulating results."""
185         results, _ = self.run_all_with_context(video_path, config)
186         return results
187
188     def config_fingerprint(self, config: Any) -> str:
189         """Stable hash of EVERYTHING that determines a validation verdict for fixed
190 video bytes.
191
192         Powers the video_id-keyed validation-result cache (backend/orchestration.py): a
193 reupload of
194 identical content reuses its stored verdict ONLY while this fingerprint still
195 matches, so any
196 change that could flip a decision correctly forces a re-validation (never serve
197 a stale
198 PASS/FAIL). What goes in ? and why each matters:
199
200 * the whole ``validation`` config block ? every threshold the validators read
201 (min_blur_threshold / min_fps / min_resolution / max_scene_cuts_per_minute /
202 pts gap /
203 relevance court-color bounds). Hashing the full model (not a hand-picked
204 subset) means a
205 NEW validation knob is covered automatically.
206 * ``sport`` ? the relevance validator keys its court-color profile off
207 ``config.sport`` (a
208 top-level field, NOT under ``validation``), so switching sports changes the
209 verdict.
210 * the sorted validator NAMES ? adding/removing a validator changes which checks
211 run.
212 * a schema version (:data:`_VALIDATION_FINGERPRINT_SCHEMA`) ? a manual escape
213 hatch for
214 validator LOGIC changes the config can't express.
215
216 Accepts a typed ``AppConfig`` (attribute access) or a plain dict;
217 ``default=str`` keeps the
218 dump total on any exotic value. Deterministic: ``sort_keys`` fixes ordering."""
219     validation: Any = getattr(config, "validation", None)
220     if validation is None and isinstance(config, dict):
221         validation = config.get("validation")
222     validation_repr = (
223         validation.model_dump(mode="json")
224         if hasattr(validation, "model_dump")
225         else validation

```

```

215         )
216         sport: Any = getattr(config, "sport", None)
217         if sport is None and isinstance(config, dict):
218             sport = config.get("sport")
219         payload = {
220             "schema": _VALIDATION_FINGERPRINT_SCHEMA,
221             "validators": sorted(v.name for v in self._validators),
222             "sport": sport,
223             "validation": validation_repr,
224         }
225         blob = json.dumps(
226             payload, sort_keys=True, default=str, separators=(",", ":")
227         )
228         return hashlib.sha256(blob.encode("utf-8")).hexdigest()
229
230
231     # Shared global registry
232     registry = ValidationRegistry()
233

```

5. user (2026-07-08T05:45:08.972Z)

```

1     from backend.pipeline.validators.base import ValidationResult, VideoValidator, registry
2     from backend.pipeline.validators.blur import BlurValidator
3     from backend.pipeline.validators.format import FormatValidator
4     from backend.pipeline.validators.pts_continuity import PTSContinuityValidator
5     from backend.pipeline.validators.relevance import RelevanceValidator
6     from backend.pipeline.validators.resolution_fps import ResolutionFPSValidator
7     from backend.pipeline.validators.scene_cut import SceneCutDensityValidator
8
9     # Register all default validators in sequence (dependencies first)
10    registry.register(FormatValidator())
11    registry.register(ResolutionFPSValidator())
12    registry.register(PTSContinuityValidator())
13    registry.register(SceneCutDensityValidator())
14    registry.register(BlurValidator())
15    registry.register(RelevanceValidator())
16
17    __all__ = ["registry", "VideoValidator", "ValidationResult"]
18

```

6. user (2026-07-08T05:45:14.947Z)

```

3:The full hash -> validate -> segment -> report pipeline (`_execute_processing`), the
69:def _validation_failure_message(failures: List[Any]) -> str:
72:     name = getattr(failure, "validator_name", None) or "validation"
76:     return "Video failed validation checks."
77:     return "Video failed validation: " + "; ".join(details)
243:     video_id: str, validation_results, segments, requested_compute
254:         "results": validation_results or [],
343:         existing.get("validation_results", []),
620:     # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
631:         skip_validation=True,
711:     """The full hash ? validate ? segment ? report pipeline for one video.
775:     # Compute-path PROVENANCE (the validation method): which backend actually ran the DETECT
?
777:     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
fail).
794:         existing_video.get("validation_results", []),
801:         #     UNLESS this exact content already has a cached verdict under the current
validator config.
804:         # reupload keys the same row. When that content was ALREADY validated under the SAME
validator
805:         # configuration, reuse the stored verdict instead of re-decoding the clip through

```

```

the whole
807:      # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
809:      # here). The fingerprint folds in the validation.* config + sport + validator set +
a schema
810:      # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811:      # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors how
it bypasses
812:      # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813:      # cloud dispatch), so a post-failure reupload reaches segmentation in ~0 validation
time.
816:      # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
818:      reused_validation = False
821:          cached = db.get_validation_cache(video_id)
829:          reused_validation = True
831:          "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
833:          "force=true re-validates.",
837:      except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to a
real validation
838:          reused_validation = False
840:          "[API] validation-cache reuse failed for video_id=%s ? re-validating.",
844:      if not reused_validation:
845:          logger.info(f"Running validations for {req.video_path}...")
848:      if not reused_validation:
849:          # Persist the verdict keyed by (content, config) so the NEXT identical reupload
skips the
850:          # decode. Best-effort (the writer swallows + logs its own faults); a miss just
re-validates.
851:          db.set_validation_cache(
859:          # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
870:          "message": _validation_failure_message(failures),
883:          # Submit-time validation already 400s unknown names; this is the defensive
957:          # Compute-path PROVENANCE stamp (the validation method): the cloud branch already
set
1121:      so the SPA always gets a clean verdict instead of a worker-crash error.""

```

7. user (2026-07-08T05:45:16.126Z)

```

202:          validation_results TEXT
267:          # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
268:          # that failed validation) lives inside `result` JSON as result["status"].
472:          # v10: validation-result CACHE keyed by the content id. A video's id IS the MD5 of
its bytes
475:          # pipeline reuses the stored verdict instead of re-decoding the clip through the
whole validator
478:          # fingerprint ? ValidationRegistry.config_fingerprint (the validation.* config +
sport +
479:          # validator set + a schema version); a lookup reuses the verdict
ONLY on a
481:          # passed ? the overall verdict (1/0), so a cached FAIL short-circuits without
a decode.
483:          # Standalone table (NO foreign key): the row is written right after validation runs
? which can
485:          # the video_id PK keeps ONE (latest) verdict per video. Additive-only ? nothing pre-
existing
488:          CREATE TABLE IF NOT EXISTS validation_cache (

```

8. user (2026-07-08T05:45:21.630Z)

```

560      except Exception as e: # noqa: BLE001 ? a staging failure is a clean job
failure, not a 500

```

```

561         logger.error(
562             "[API] cloud-serving: failed to stage %s -> %s: %s",
563             local_src,
564             staged_uri,
565             e,
566             exc_info=True,
567         )
568         return _failed(
569             f"cloud-serving: failed to stage the uploaded file to {input_root}:
{e}",
570             False,
571         )
572         # Repoint BOTH the dispatch and the DB video record at the staged bucket object:
the L4 job
573         # DETECTs from it, and a later /api/compile re-fetches the same source (the
local upload is
574         # gone once this instance recycles). add_video is an UPSERT ? it only updates
filepath.
575         req.video_path = staged_uri
576         input_ref = storage.resolve_input_ref(staged_uri, storage_cfg)
577         db.add_video(
578             video_id, staged_uri, "processed", [r.model_dump() for r in val_results]
579         )
580         out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
581         if not out_root:
582             return _failed(
583                 "cloud-serving needs storage.output_root set to a bucket (e.g. gs://your-
bucket) "
584                 "? that's where the Cloud Run job writes the result.",
585                 False,
586             )
587
588         notify("dispatching (cloud_run)")
589         logger.info(
590             "[API] cloud-serving: dispatching %s -> Cloud Run Job (out=%s)",
591             req.video_path,
592             out_root,
593         )
594         # Forward the engine's AI-handoff stance so the cloud job runs the SAME detection:
with
595         # skip_ai_handoff=True the WASB windows become rallies directly (no Gemini key
needed); without
596         # forwarding the cloud job would default to the handoff and yield 0 rallies when it
has no key.
597         # (player_pool / max_frames are still dropped on the cloud path ? a separate
documented follow-up.)
598         overrides = []
599         idx_cfg = getattr(cfg, "indexing", None)
600         sk = getattr(idx_cfg, "skip_ai_handoff", None)
601         if sk is not None:
602             overrides.append(f"indexing.skip_ai_handoff={'true' if sk else 'false'}")
603         # Forward the resolved WASB serving knobs so the PRESET governs the worker
(#506/#507). The
604         # worker Job has no DEPLOYMENT_PROFILE, so it does NOT apply the cloud-serving
preset itself ?
605         # without forwarding, its decode_backend + GPU-feed tuning silently fall back to the
model
606         # defaults (cpu, batch 8) regardless of what the control-plane resolved.
decode_backend is
607         # cuda-only (validator-enforced) and requires a worker image that understands
--decode-backend
608         # (? #507); batch_size/prefetch_batches are forwarded only when set (None = the
parity default).
609         wasb_cfg = getattr(idx_cfg, "wasb", None)
610         if wasb_cfg is not None:

```



```

611         decode_backend = getattr(wasb_cfg, "decode_backend", None)
612         if decode_backend:
613             overrides.append(f"indexing.wasb.decode_backend={decode_backend}")
614         batch_size = getattr(wasb_cfg, "batch_size", None)
615         if batch_size:
616             overrides.append(f"indexing.wasb.batch_size={batch_size}")
617         prefetch = getattr(wasb_cfg, "prefetch_batches", None)
618         if prefetch:
619             overrides.append(f"indexing.wasb.prefetch_batches={prefetch}")
620         # Tell the worker to SKIP its redundant re-validation: THIS control-plane is the
authoritative
621         # gate ? it already ran the validators with its resolved config just above (only a
PASS reaches
622         # here; a failure returned "failed" before dispatch). A worker re-run would decode
the video a
623         # second time and, because its config isn't fully forwarded, can resolve DIFFERENT
thresholds and
624         # REJECT a clip this gate already passed (observed 2026-07-07: passed at
min_blur_threshold=8.0,
625         # then worker-rejected at its default 20.0 after a successful GPU dispatch).
626         spec = ComputeJobSpec(
627             video_uri=req.video_path,
628             out_uri=out_root,
629             segmenter=seg_name,
630             config_overrides=tuple(overrides),
631             skip_validation=True,
632         )
633         try:
634             # Dispatch the Cloud Run Job + bucket-poll to completion. on_wait is the live
635             # heartbeat (#482): job.progress moves on every poll tick instead of freezing at
636             # "segmenting" for the whole GPU run (the 2026-07-05 silent 22-min spinner) ?
which
637             # also gives the SPA a stall signal to time out on, instead of a blind wall
clock.
638             result = compute.run_infer(
639                 spec,
640                 on_wait=lambda elapsed: notify(_wait_progress_message(seg_name, elapsed)),
641             )
642         except Exception as e: # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
643             logger.error("[API] cloud dispatch error: %s", e, exc_info=True)
644             return _failed(f"cloud dispatch error: {e}", True)
645         if result.returncode != 0:
646             return _failed(f"cloud job failed (returncode {result.returncode}).", True)
647         if result.video_id and result.video_id != video_id:
648             # The worker re-hashes the bucket bytes; a divergence means the input changed
between this
649             # box's hash and the worker's ? ingest under the LOCAL id but flag it so it's
diagnosable.
650             logger.warning(
651                 "[API] cloud video_id %s != local %s ? input bytes differ between the API
hash "
652                 "and the worker hash; ingesting under the local id.",
653                 result.video_id,
654                 video_id,
655             )
656
657         notify("ingesting (cloud_run)")
658         storage_dict = (
659             storage_cfg.model_dump() # type: ignore[union-attr] # hasattr-guarded; mypy
can't narrow
660             if hasattr(storage_cfg, "model_dump")
661             else {}
662         )
663         try:

```

```

664         _ingest_remote_segments(db, video_id, result.outputs_uri, storage_dict)
665     except Exception as e: # noqa: BLE001 ? a missing/partial/corrupt result must not
500 or leak rows
666         logger.error("[API] cloud ingest failed: %s", e, exc_info=True)
667         return _failed(
668             f"cloud-serving: could not read the result from {result.outputs_uri}: {e}",
669             True,
670         )
671
672     # Only the NEW cloud rows (id > the pre-run watermark); _finalize_reingest then
drops the old ones.
673     segments = [
674         s for s in db.query_play_segments(video_id, {}) if int(s.get("id", 0)) > play_wm
675     ]
676     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
677     # Compute-path PROVENANCE: the GCP-verifiable proof the cloud path ran. `execution`
is the real
678     # Cloud Run execution name (cross-check: gcloud run jobs executions describe
<execution>).
679     provenance = {
680         "path": "cloud_run",
681         "requested": req.compute,
682         "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
683         "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
684         "execution": result.execution,
685         "outputs_uri": result.outputs_uri,
686     }
687     logger.info(
688         "[COMPUTE] cloud_run dispatched: execution=%s job=%s region=%s out=%s
video_id=%s",
689         result.execution,
690         provenance["job"],
691         provenance["region"],
692         result.outputs_uri,
693         video_id,
694     )
695     return (
696         {
697             "video_id": video_id,
698             "status": "success",
699             "message": f"Successfully indexed {len(segments)} play segments (cloud_run /
'{seg_name}').",
700             "results": [r.model_dump() for r in val_results],
701             "play_segments": segments,
702             "compute": provenance,
703         },
704         True,
705     )
706
707
708     def _execute_processing(
709         config=None, db=None, req=None, progress=None
710     ) -> Dict[str, Any]:
711         """The full hash ? validate ? segment ? report pipeline for one video.
712
713         Accepts explicit ``config``/``db`` (the new app-factory pattern) OR reads the
714         module-level ``global_config``/``db_instance`` (backward-compatible with existing
715         tests that pass only ``req``). Route handlers always pass explicit params; older
716         test code that calls ``_execute_processing(req)`` still works.
717
718         ``progress`` is an optional callable(stage: str) used to surface coarse job
progress.
719         Raises on unexpected internal errors ? the caller records those as a job failure
720         """
721         import backend.main as _main # call-time seam: main's globals/model stay patchable

```

```

722
723     # Backward-compatible shim: if called as _execute_processing(req) (old pattern),
724     # shift positional args ? req arrives in position 0 (config), progress in position 1
725     (db).
726     if config is not None and not isinstance(config, AppConfig):
727         # Old signature: _execute_processing(req, progress=None)
728         if req is None and db is not None and isinstance(db, _main.ProcessRequest):
729             req = db
730             db = None
731         elif req is None and isinstance(config, _main.ProcessRequest):
732             req = config
733             config = None
734     # Resolve config/db from module globals when not passed explicitly
735     if config is None:
736         config = _main.global_config
737     if db is None:
738         db = _main.db_instance
739
740     notify = progress or (lambda stage: None)
741
742     notify("hashing")
743     # M2: resolve the input to a LOCAL path. A bare/local path is returned unchanged
744     (identity ?
745     # byte-identical to today); a bucket/Drive URI is fetched to scratch first. The
746     original
747     # req.video_path (the source URI) is kept for the DB record + report; only the file-
748     reading
749     # consumers (hash / validators / segmenter) use the resolved local path.
750     local_video = ""
751     try:
752         local_video = storage.acquire_local_input(
753             req.video_path, getattr(config, "storage", None)
754         )
755         video_id = _main.compute_video_id(local_video)
756     except Exception:
757         storage.cleanup_acquired_local_input(
758             req.video_path, local_video, getattr(config, "storage", None)
759         )
760         raise
761     try:
762         existing_video = db.get_video(video_id)
763     except Exception:
764         storage.cleanup_acquired_local_input(
765             req.video_path, local_video, getattr(config, "storage", None)
766         )
767         raise
768     was_reingest = existing_video is not None
769
770     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
771     default_seg = getattr(
772         getattr(config, "indexing", None), "default_segmenter", "motion"
773     )
774     seg_name = req.segmenter or default_seg
775     segmenter_actual = None
776     segmentation_ran = False
777     val_results: List[Any] = []
778     val_context: Dict[str, Any] = {}
779     # Compute-path PROVENANCE (the validation method): which backend actually ran the
780     DETECT ?
781     # "in_process" once the local segmenter is reached, set on the response as
782     "cloud_run" by
783     # _maybe_dispatch_remote when it dispatches. None ? nothing ran (validation/config
784     fail).
785     compute_actual: Optional[str] = None
786     response: Optional[Dict[str, Any]] = None

```

```

780         try:
781             if (
782                 existing_video
783                 and existing_video.get("status") == "processed"
784                 and not req.force
785             ):
786                 cached_segments = db.query_play_segments(video_id, {})
787                 if cached_segments:
788                     logger.info(
789                         "[API] Reusing cached segmentation for video_id=%s; force=true
bypasses cache.",
790                         video_id,
791                     )
792                 response = _cached_process_payload(
793                     video_id,
794                     existing_video.get("validation_results", []),
795                     cached_segments,
796                     getattr(req, "compute", None),
797                 )
798                 return response
799
800             # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for
the report) ?
801             #     UNLESS this exact content already has a cached verdict under the current
validator config.
802             notify("validating")
803             # VALIDATION-RESULT CACHE: video_id is the MD5 of the file bytes, so an
identical-content
804             # reupload keys the same row. When that content was ALREADY validated under the
SAME validator
805             # configuration, reuse the stored verdict instead of re-decoding the clip
through the whole
806             # validator suite ? minutes on a heavy 4K clip on the CPU-only control-plane,
which the
807             # 2026-07-07 CUJ re-paid on every retry after a segmentation failure (validation
passed but
808             # detection failed ? no cached segments, so the whole-pipeline cache above
misses and we land
809             # here). The fingerprint folds in the validation.* config + sport + validator
set + a schema
810             # version, so ANY config change misses and re-validates (never serve a stale
PASS/FAIL).
811             # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
812             # the whole-pipeline cache). Composes with #513 (the worker skips its own re-
validation on a
813             # cloud dispatch), so a post-failure reupload reaches segmentation in ~0
validation time.
814             val_fingerprint = registry.config_fingerprint(config)
815             # The whole read ? fingerprint-match ? replay is guarded: ANY cache fault
(unreadable row,
816             # corrupt or schema-incompatible stored results) degrades to a real validation
rather than
817             # failing the job. The cache is a pure speedup ? never a new failure mode.
818             reused_validation = False
819             if not req.force:
820                 try:
821                     cached = db.get_validation_cache(video_id)
822                     if cached is not None and cached.get("fingerprint") == val_fingerprint:
823                         # Replay the stored ValidationResult list; val_context stays {} (no
fresh
824                         # ffprobe_meta ? the report's media block degrades gracefully to
size-only, the
825                         # deliberate cost of skipping the decode). Segmentation reads
neither of these.

```

```

826         val_results = [
827             ValidationResult(**r) for r in cached.get("results", [])
828         ]
829         reused_validation = True
830         logger.info(
831             "[API] Reusing cached validation verdict (passed=%s) for
video_id=%s; "
832             "validator config unchanged ? skipping the validator suite. "
833             "force=true re-validates.",
834             cached.get("passed"),
835             video_id,
836         )
837     except Exception: # noqa: BLE001 ? any cache read/replay fault degrades to
a real validation
838         reused_validation = False
839         logger.warning(
840             "[API] validation-cache reuse failed for video_id=%s ? re-
validating.",
841             video_id,
842             exc_info=True,
843         )
844     if not reused_validation:
845         logger.info(f"Running validations for {req.video_path}...")
846         val_results, val_context = registry.run_all_with_context(local_video,
config)
847         failures = [r for r in val_results if not r.passed]
848         if not reused_validation:
849             # Persist the verdict keyed by (content, config) so the NEXT identical
reupload skips the
850             # decode. Best-effort (the writer swallows + logs its own faults); a miss
just re-validates.
851             db.set_validation_cache(
852                 video_id,
853                 val_fingerprint,
854                 not failures,
855                 [r.model_dump() for r in val_results],
856             )
857
858         if failures:
859             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
860             # status; it no longer destroys the video's prior good segments.
861             db.add_video(
862                 video_id,
863                 req.video_path,
864                 "failed",
865                 [r.model_dump() for r in val_results],
866             )
867             response = {
868                 "video_id": video_id,
869                 "status": "failed",
870                 "message": _validation_failure_message(failures),
871                 "results": [r.model_dump() for r in val_results],
872             }
873             return response
874
875     # 2. Run segmenter & indexer
876     logger.info("[API] Video passed checks. Segmenting and indexing...")
877     db.add_video(
878         video_id, req.video_path, "processed", [r.model_dump() for r in val_results]
879     )
880
881     segmenter_cls = segmenter_registry.get(seg_name)
882     if not segmenter_cls:
883         # Submit-time validation already 400s unknown names; this is the defensive
884         # in-worker path (e.g. config default pointing at an unregistered

```

```

segmenter).
885         available = list(segmenter_registry.list_available().keys())
886         response = {
887             "video_id": video_id,
888             "status": "failed",
889             "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
890             "results": [r.model_dump() for r in val_results],
891             "play_segments": [],
892         }
893         return response
894
895         logger.info(f"[API] Using segmenter: {seg_name}")
896         notify(f"segmenting ({seg_name})")
897         segmenter_actual = seg_name
898         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
899         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
900         play_wm, cand_wm = db.segment_watermarks(video_id)
901         # COMPUTE_DECOUPLED_SERVING M6 ? API: with deployment.compute_target=cloud_run,
run the heavy
902         # DETECT on a Cloud Run GPU Job + ingest its rallies HERE so query/compile are
unchanged.
903         # DEFAULT-OFF: returns None for every non-cloud target ? the in-process
segmenter below runs
904         # byte-identically. (player_pool/max_frames aren't threaded to the worker
`infer` CLI yet, so
905         # they're dropped on the cloud path in this slice ? a documented follow-up.)
906         remote = _maybe_dispatch_remote(
907             config, db, req, video_id, seg_name, val_results, play_wm, cand_wm, notify
908         )
909         if remote is not None:
910             response, segmentation_ran = (
911                 remote # `ran` = did the cloud job actually execute (report)
912             )
913             return response # the cloud branch already stamped response["compute"]
(path=cloud_run)
914         # In-process from here on ? record the actual path for the provenance stamp in
`finally`.
915         compute_actual = "in_process"
916         segmenter = segmenter_cls(db, config)
917         try:
918             # Serialize the CPU/GPU-heavy in-process detection (see
_LOCAL_COMPUTE_LANE). A remote
919             # dispatch never reaches here (it returned above), so remote polls still
overlap freely.
920             with _LOCAL_COMPUTE_LANE:
921                 result = segmenter.process_video(
922                     video_id, local_video, req.player_pool, req.max_frames
923                 )
924         except Exception:
925             # A raising segmenter must not leave half-written rows: restore the pre-run
926             # state (same destroy-after-confirm contract as the Failure branch), then
let
927             # the job worker record the failure.
928             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
929             raise
930         segmentation_ran = True
931
932         if isinstance(result, Failure):
933             logger.error(f"[API] Segmenter failed: {result.message}")
934             db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
935             response = {
936                 "video_id": video_id,

```

```

937         "status": "failed",
938         "message": f"Segmenter '{seg_name}' failed: {result.message}",
939         "results": [r.model_dump() for r in val_results],
940         "play_segments": [],
941     }
942     return response
943
944     segments = result.value if hasattr(result, "value") else result
945     notify("finalizing")
946     _finalize_reingest(db, video_id, segments, play_wm, cand_wm)
947
948     response = {
949         "video_id": video_id,
950         "status": "success",
951         "message": f"Successfully indexed {len(segments)} play segments using
952 '{seg_name}' segmenter.", # type: ignore[arg-type] # hasattr-unwrap above; Success arm has
953         "results": [r.model_dump() for r in val_results],
954         "play_segments": segments,
955     }
956     return response
957 finally:
958     # Compute-path PROVENANCE stamp (the validation method): the cloud branch
959     already set
960     # response["compute"] (path=cloud_run + the Cloud Run execution); for every
961     other path stamp
962     # the actual path here so the job result PROVES which backend ran (in_process /
963     not_run) and
964     # what the picker REQUESTED ? a silent fallback (requested cloud, ran local) is
965     then visible.
966     if isinstance(response, dict):
967         response.setdefault(
968             "compute",
969             {
970                 "path": compute_actual or "not_run",
971                 "requested": getattr(req, "compute", None),
972             },
973         )
974     prov = response.get("compute", {}) if isinstance(response, dict) else {}
975     logger.info(
976         "[COMPUTE] video_id=%s requested=%s actual=%s dispatched=%s status=%s",
977         video_id,
978         getattr(req, "compute", None),
979         prov.get("path"),
980         prov.get("dispatched"),
981         (response or {}).get("status") if isinstance(response, dict) else None,
982     )
983     # Always emit the per-video observability report (next to the video, or to
984     # report.output_dir). Never raises. Surface the paths in the response. (The
985     compute-path
986     # proof lives on the job result + the [COMPUTE] log, not the report ?
987     obsreport's record_run
988     # doesn't surface arbitrary run signals.)
989     paths = _main.emit_indexer_report(
990         db=db,
991         video_id=video_id,
992         # Resolved LOCAL path, not the raw URI: obsreport writes the report next to
993         the
994         # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://,
995         local://)
996         # collapses to a literal scheme dir at the repo root (see #397). The source
997         URI is
998         # still preserved in DB provenance via db.add_video.
999         video_path=local_video,
1000         config=_report_config_for_input(config, req.video_path),

```

```

991         val_results=val_results,
992         val_context=val_context,
993         segmenter_requested=req.segmenter,
994         segmenter_actual=segmenter_actual,
995         was_reingest=was_reingest,
996         segmentation_ran=segmentation_ran,
997     )
998     if paths and isinstance(response, dict):
999         response["reports"] = paths

```

9. user (2026-07-08T05:45:30.629Z)

```

1     """Shared single-pass frame sampler for the frame-decoding validators.
2
3     Historically ``scene_cut``, ``blur`` and ``relevance`` each opened their own
4     ``cv2.VideoCapture`` and pulled every sample frame via random access
5     (``cap.set(cv2.CAP_PROP_POS_FRAMES, idx)`` then ``cap.read()``). On long-GOP
6     HEVC / 4K footage (GoPro) decoded on a CPU-only control-plane, every
7     ``POS_FRAMES`` seek forces FFmpeg to jump to the nearest keyframe and decode
8     forward, so ~125 seeks across three independent decode opens cost minutes
9     (observed 6m47s "Running validations" on a 1.41 GiB clip before segmentation
10    even started).
11
12    This module decodes the same sample indices in a single open with one
13    monotonic-forward pass:
14
15    * ``cap.grab()`` advances cheaply through the small gaps between the dense
16    ``scene_cut`` samples (which cluster in the first ~50s), and
17    * a single forward ``POS_FRAMES`` seek skips the large gaps between the sparse
18    whole-video ``blur`` / ``relevance`` samples (a bounded keyframe jump instead
19    of decoding everything in between).
20
21    The frames handed to each validator are byte-identical to the old per-validator
22    seeks (verified against the reference clip), so pass/fail decisions are
23    unchanged ? only the wall-clock collapses. Frames are streamed: at most one
24    decoded frame is held in memory at a time (each frame is dispatched to the
25    interested validators' online accumulators and then dropped), so a 4K clip does
26    not balloon RAM the way caching ~125 full-resolution frames would.
27
28    Why a hybrid (grab + occasional forward seek) rather than pure ``grab()``:
29    ``blur``/``relevance`` sample right up to the end of the file, so a pure
30    forward ``grab()`` pass would decode the whole video ? measured slower
31    than the seek-per-sample status quo on the 1.4 GiB reference clip (38s vs 27s).
32    The hybrid grabs only the dense region and forward-seeks the rest: ~8.6s on the
33    same clip, with 17 monotonic-forward seeks instead of 126 random ones.
34    """
35
36    from __future__ import annotations
37
38    import logging
39    from collections import defaultdict
40    from dataclasses import dataclass
41    from typing import Any, Dict, List, Sequence, Tuple
42
43    import cv2
44
45    logger = logging.getLogger(__name__)
46
47    # Context keys populated by the registry's shared pre-pass and consumed by each
48    # FrameSamplingValidator.validate().
49    FRAME_META_KEY = "frame_sampler_meta"
50    FRAME_STATES_KEY = "frame_sampler_states"
51
52    # Grab through gaps up to ~2 seconds of frames (>= a long GoPro GOP); forward-seek
53    # anything larger. Below this a seek+keyframe-decode costs more than just grabbing

```



```

54 # the intervening frames, and it keeps the dense scene_cut region (samples ~0.5s
55 # apart) on the cheap sequential path.
56 _SEEK_GAP_FLOOR_FRAMES = 64
57
58
59 @dataclass
60 class SampleMeta:
61     """Decode metadata read once from the shared capture (same source the old
62     per-validator opens used: ``cv2.VideoCapture.get``)."""
63
64     opened: bool
65     fps: float
66     total_frames: int
67
68
69 def _seek_gap_frames(fps: float) -> int:
70     """Frame-gap threshold above which we forward-seeking instead of grabbing."""
71     if fps and fps > 0:
72         return max(int(round(fps)) * 2, _SEEK_GAP_FLOOR_FRAMES)
73     return _SEEK_GAP_FLOOR_FRAMES
74
75
76 def run_frame_sampling(
77     video_path: str, samplers: Sequence[Any], config: Any
78 ) -> Tuple[SampleMeta, Dict[str, Any]]:
79     """Decode ``video_path`` once and feed each sampler's online accumulator.
80
81     ``samplers`` are FrameSamplingValidator instances (duck-typed here to avoid a
82     circular import): each provides ``name``, ``sample_indices(total, fps)`,
83     ``_init_state(config)`` and ``_consume(state, frame_idx, frame)``.
84
85     Returns ``(meta, states)`` where ``states`` maps ``validator.name`` to the
86     accumulator each validator later ``_finalize`` s. States are always
87     initialised (even when the file can't be opened) so finalize can render the
88     right "failed to open" / "invalid frame count" message.
89     """
90     states: Dict[str, Any] = {s.name: s._init_state(config) for s in samplers}
91
92     cap = cv2.VideoCapture(video_path)
93     try:
94         if not cap.isOpened():
95             return SampleMeta(opened=False, fps=0.0, total_frames=0), states
96
97         fps = float(cap.get(cv2.CAP_PROP_FPS) or 0.0)
98         total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
99         meta = SampleMeta(opened=True, fps=fps, total_frames=total_frames)
100         if total_frames <= 0:
101             return meta, states
102
103         # Which validators want each frame index (multiple may share an index,
104         # e.g. frame 0). Sorted union drives one monotonic-forward pass.
105         wanted: Dict[int, List[Any]] = defaultdict(list)
106         for sampler in samplers:
107             for idx in sampler.sample_indices(total_frames, fps):
108                 wanted[idx].append(sampler)
109
110         _decode_and_dispatch(cap, sorted(wanted), wanted, states, fps)
111         return meta, states
112     finally:
113         cap.release()
114
115
116 def _decode_and_dispatch(
117     cap: Any,
118     order: List[int],

```

```

119         wanted: Dict[int, List[Any]],
120         states: Dict[str, Any],
121         fps: float,
122     ) -> None:
123         """One monotonic-forward pass; retrieve only at wanted indices and dispatch."""
124         gap = _seek_gap_frames(fps)
125         pos = 0
126         for target in order:
127             if target - pos > gap:
128                 # Skip a large gap with a single forward seek (bounded keyframe jump).
129                 cap.set(cv2.CAP_PROP_POS_FRAMES, target)
130                 pos = target
131             else:
132                 # Grab (decode, no copy) forward through the small gap.
133                 while pos < target:
134                     if not cap.grab():
135                         return # EOF: no further sample index is reachable.
136                     pos += 1
137
138             ok, frame = cap.read() # grab + retrieve the wanted frame at pos == target
139             if not ok:
140                 return # EOF / undecodable tail: remaining indices are unreachable.
141             for sampler in wanted[target]:
142                 sampler._consume(states[sampler.name], target, frame)
143             pos = target + 1
144

```

10. user (2026-07-08T05:45:30.865Z)

```

1     import json
2     import os
3     import subprocess
4     from typing import Any
5
6     from backend.pipeline.validators.base import ValidationResult, VideoValidator
7     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
8         ffprobe_bin,
9         ffprobe_timeout_sec,
10        timeout_label,
11    )
12
13
14    class FormatValidator(VideoValidator):
15        @property
16        def name(self) -> str:
17            return "format_check"
18
19        @property
20        def description(self) -> str:
21            return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
22
23        def validate(
24            self, video_path: str, config: Any, context: dict[str, Any]
25        ) -> ValidationResult:
26            if not os.path.exists(video_path):
27                return ValidationResult(
28                    validator_name=self.name,
29                    passed=False,
30                    message=f"File not found: {video_path}",
31                )
32
33            cmd = [
34                ffprobe_bin(),
35                "-v",

```

```

36         "error",
37         "-select_streams",
38         "v:0",
39         "-show_format",
40         "-show_streams",
41         "-of",
42         "json",
43         video_path,
44     ]
45
46     try:
47         timeout = ffprobe_timeout_sec()
48         result = subprocess.run(
49             cmd, capture_output=True, text=True, check=True, timeout=timeout
50         )
51         info = json.loads(result.stdout)
52
53         if not info.get("streams") or not info.get("format"):
54             return ValidationResult(
55                 validator_name=self.name,
56                 passed=False,
57                 message="Failed to retrieve valid video streams from the file.",
58             )
59
60         # Store the metadata in context for subsequent validators to utilize
61         context["ffprobe_meta"] = info
62
63         stream = info["streams"][0]
64         fmt = info["format"]
65
66         details = {
67             "format_name": fmt.get("format_name"),
68             "duration": float(fmt.get("duration", 0)),
69             "size_bytes": int(fmt.get("size", 0)),
70             "codec_name": stream.get("codec_name"),
71         }
72
73         # Check if container name includes mp4
74         if "mp4" not in fmt.get("format_name", "").lower():
75             return ValidationResult(
76                 validator_name=self.name,
77                 passed=False,
78                 message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
79                 details=details,
80             )
81
82         return ValidationResult(
83             validator_name=self.name,
84             passed=True,
85             message="Video format is valid MP4 container.",
86             details=details,
87         )
88
89     except subprocess.CalledProcessError as e:
90         return ValidationResult(
91             validator_name=self.name,
92             passed=False,
93             message="Video stream is corrupted or unreadable by ffprobe.",
94             details={"error": e.stderr},
95         )
96     except subprocess.TimeoutExpired:
97         return ValidationResult(
98             validator_name=self.name,
99             passed=False,

```

```

100             message=f"ffprobe timed out after
{timeout_label(ffprobe_timeout_sec())}.",
101         )
102     except Exception as e:
103         return ValidationResult(
104             validator_name=self.name,
105             passed=False,
106             message=f"Unexpected error running ffprobe: {str(e)}",
107         )
108

```

11. user (2026-07-08T05:45:36.269Z)

```

1     from typing import Any, List
2
3     import cv2
4
5     from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
6     from backend.pipeline.validators.frame_sampler import SampleMeta
7
8     # Correlation histogram threshold: 1.0 is a perfect match; < 0.6 flags a sudden
9     # massive scene change (a cut).
10    _CUT_THRESHOLD = 0.6
11    # Cap the number of samples so validation stays instant on long clips (the
12    # samples cover roughly the first 50 seconds at 2 fps).
13    _MAX_SAMPLES = 100
14
15
16    class SceneCutDensityValidator(FrameSamplingValidator):
17        @property
18        def name(self) -> str:
19            return "scene_cut_check"
20
21        @property
22        def description(self) -> str:
23            return "Analyzes visual frames to detect abrupt scene cuts, which indicate an
edited or compiled video."
24
25        def sample_indices(self, total_frames: int, fps: float) -> List[int]:
26            # Sample at 2 frames per second (fps/2 step), capped at _MAX_SAMPLES.
27            sample_interval = max(1, int(fps / 2))
28            indices: List[int] = []
29            frame_idx = 0
30            sampled_count = 0
31            while True:
32                indices.append(frame_idx)
33                sampled_count += 1
34                frame_idx += sample_interval
35                if frame_idx >= total_frames or sampled_count > _MAX_SAMPLES:
36                    break
37            return indices
38
39        def _init_state(self, config: Any) -> dict[str, Any]:
40            return {"cuts_detected": 0, "sampled_count": 0, "prev_hist": None}
41
42        def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
43            # Calculate color histogram (hue/saturation) and compare to the previous
44            # sample; a big correlation drop is a cut.
45            hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
46            hist = cv2.calcHist([hsv], [0, 1], None, [50, 60], [0, 180, 0, 256])
47            cv2.normalize(hist, hist, alpha=0, beta=1, norm_type=cv2.NORM_MINMAX)
48
49            prev_hist = state["prev_hist"]
50            if prev_hist is not None:
51                score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_CORREL)

```

```

52         if score < _CUT_THRESHOLD:
53             state["cuts_detected"] += 1
54
55         state["prev_hist"] = hist
56         state["sampled_count"] += 1
57
58     def _finalize(
59         self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
60     ) -> ValidationResult:
61         if not meta.opened:
62             return ValidationResult(
63                 validator_name=self.name,
64                 passed=False,
65                 message="Failed to open video file using OpenCV.",
66             )
67
68         fps = meta.fps
69         total_frames = meta.total_frames
70         duration = total_frames / fps if fps > 0 else 0
71
72         if duration <= 0:
73             return ValidationResult(
74                 validator_name=self.name,
75                 passed=False,
76                 message="Invalid video duration.",
77             )
78
79         sample_interval = max(1, int(fps / 2))
80         cuts_detected = state["cuts_detected"]
81         sampled_count = state["sampled_count"]
82
83         # Compute cuts per minute
84         sampled_duration = (sampled_count * sample_interval) / fps if fps > 0 else 1
85         cuts_per_minute = (
86             (cuts_detected / sampled_duration) * 60 if sampled_duration > 0 else 0
87         )
88
89         details = {
90             "cuts_detected": cuts_detected,
91             "sampled_frames": sampled_count,
92             "cuts_per_minute": round(cuts_per_minute, 3),
93         }
94
95         val_cfg = config.validation
96         max_cuts_rate = val_cfg.max_scene_cuts_per_minute
97
98         if cuts_per_minute > max_cuts_rate:
99             return ValidationResult(
100                 validator_name=self.name,
101                 passed=False,
102                 message=f"Video splicing compromise: Detected {cuts_detected} cuts
103                 ({round(cuts_per_minute, 2)} cuts/min), which exceeds the continuous raw file limit of
104                 {max_cuts_rate} cuts/min.",
105                 details=details,
106             )
107
108         return ValidationResult(
109             validator_name=self.name,
110             passed=True,
111             message="Scene continuity check passed. No abnormal visual cuts detected.",
112             details=details,
113         )

```

```

1      from typing import Any, List
2
3      import cv2
4
5      from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
6      from backend.pipeline.validators.frame_sampler import SampleMeta
7
8      # Number of frames sampled evenly across the whole video.
9      _NUM_SAMPLES = 15
10
11
12     class BlurValidator(FrameSamplingValidator):
13         @property
14         def name(self) -> str:
15             return "blur_check"
16
17         @property
18         def description(self) -> str:
19             return "Samples frames across the video and checks if the footage is excessively
20 blurry or out of focus."
21
22     def sample_indices(self, total_frames: int, fps: float) -> List[int]:
23         # Sample _NUM_SAMPLES frames evenly distributed across the video.
24         step = max(1, total_frames // _NUM_SAMPLES)
25         return [i * step for i in range(_NUM_SAMPLES)]
26
27     def _init_state(self, config: Any) -> dict[str, Any]:
28         return {"variances": []}
29
30     def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
31         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
32         # Calculate the Laplacian variance (standard focus metric)
33         laplacian_var = cv2.Laplacian(gray, cv2.CV_64F).var()
34         state["variances"].append(laplacian_var)
35
36     def _finalize(
37         self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
38     ) -> ValidationResult:
39         if not meta.opened:
40             return ValidationResult(
41                 validator_name=self.name,
42                 passed=False,
43                 message="Failed to open video file for blur analysis.",
44             )
45
46         if meta.total_frames <= 0:
47             return ValidationResult(
48                 validator_name=self.name, passed=False, message="Invalid frame count."
49             )
50
51         variances = state["variances"]
52         if not variances:
53             return ValidationResult(
54                 validator_name=self.name,
55                 passed=False,
56                 message="Could not read frames to perform blur verification.",
57             )
58
59         avg_variance = sum(variances) / len(variances)
60
61         details = {
62             "avg_laplacian_variance": round(avg_variance, 2),
63             "min_laplacian_variance": round(min(variances), 2),
64             "max_laplacian_variance": round(max(variances), 2),
65             "sampled_frames_count": len(variances),

```

```

65         }
66
67         val_cfg = config.validation
68         min_blur_threshold = val_cfg.min_blur_threshold
69
70         if avg_variance < min_blur_threshold:
71             return ValidationResult(
72                 validator_name=self.name,
73                 passed=False,
74                 message=f"Video quality rejected: Focus check failed. Average sharpness
score of {round(avg_variance, 1)} is below the minimum required threshold of
{min_blur_threshold}.",
75                 details=details,
76             )
77
78         return ValidationResult(
79             validator_name=self.name,
80             passed=True,
81             message="Video sharpness meets focus requirements.",
82             details=details,
83         )
84

```

13. user (2026-07-08T05:45:42.261Z)

```

1     from typing import Any, List
2
3     import cv2
4     import numpy as np
5
6     from backend.pipeline.validators.base import FrameSamplingValidator, ValidationResult
7     from backend.pipeline.validators.frame_sampler import SampleMeta
8
9     # Number of frames sampled evenly across the whole video.
10    _NUM_SAMPLES = 10
11
12
13    class RelevanceValidator(FrameSamplingValidator):
14        @property
15        def name(self) -> str:
16            return "relevance_check"
17
18        @property
19        def description(self) -> str:
20            return "Checks if the video content is relevant to badminton by scanning for
court color palettes and geometric features."
21
22        def sample_indices(self, total_frames: int, fps: float) -> List[int]:
23            # Sample _NUM_SAMPLES frames across the video.
24            step = max(1, total_frames // _NUM_SAMPLES)
25            return [i * step for i in range(_NUM_SAMPLES)]
26
27        def _init_state(self, config: Any) -> dict[str, Any]:
28            # Resolve the court-color bounds ONCE per run (config wins over the sport
29            # profile, Pydantic defaults as fallback) ? same precedence the old
30            # per-frame loop applied on every iteration.
31            sport_name = config.sport
32            from backend.sports import sport_registry
33
34            try:
35                sport_profile = sport_registry.get(sport_name)
36                sport_rel_cfg = sport_profile.get_relevance_config()
37            except KeyError:
38                sport_rel_cfg = {}
39

```

```

40         rel_cfg = config.validation.relevance
41
42         hsv_lower = np.array(
43             rel_cfg.court_green_lower
44             or sport_rel_cfg.get("court_green_lower")
45             or [35, 40, 40]
46         )
47         hsv_upper = np.array(
48             rel_cfg.court_green_upper
49             or sport_rel_cfg.get("court_green_upper")
50             or [85, 255, 255]
51         )
52         min_court_ratio = (
53             rel_cfg.court_pixels_min_ratio
54             if rel_cfg.court_pixels_min_ratio is not None
55             else sport_rel_cfg.get("court_pixels_min_ratio", 0.05)
56         )
57
58         return {
59             "hsv_lower": hsv_lower,
60             "hsv_upper": hsv_upper,
61             "min_court_ratio": min_court_ratio,
62             "green_ratios": [],
63         }
64
65     def _consume(self, state: dict[str, Any], frame_idx: int, frame: Any) -> None:
66         # Downscale frame for speed, then measure the court-green pixel ratio.
67         small_frame = cv2.resize(frame, (320, 240))
68         hsv = cv2.cvtColor(small_frame, cv2.COLOR_BGR2HSV)
69
70         mask = cv2.inRange(hsv, state["hsv_lower"], state["hsv_upper"])
71         green_pixels = cv2.countNonZero(mask)
72         total_pixels = mask.shape[0] * mask.shape[1]
73
74         state["green_ratios"].append(green_pixels / total_pixels)
75
76     def _finalize(
77         self, state: dict[str, Any], meta: SampleMeta, config: Any, context: dict
78     ) -> ValidationResult:
79         if not meta.opened:
80             return ValidationResult(
81                 validator_name=self.name,
82                 passed=False,
83                 message="Failed to open video file for relevance validation.",
84             )
85
86         if meta.total_frames <= 0:
87             return ValidationResult(
88                 validator_name=self.name, passed=False, message="Invalid frame count."
89             )
90
91         green_ratios = state["green_ratios"]
92         if not green_ratios:
93             return ValidationResult(
94                 validator_name=self.name,
95                 passed=False,
96                 message="Failed to extract frames for relevance checking.",
97             )
98
99         sport_name = config.sport
100         min_court_ratio = state["min_court_ratio"]
101         avg_green_ratio = sum(green_ratios) / len(green_ratios)
102
103         details = {
104             "avg_green_court_ratio": round(avg_green_ratio, 4),

```



```

105         "max_green_court_ratio": round(max(green_ratios), 4),
106         "min_green_court_ratio": round(min(green_ratios), 4),
107         "green_threshold": min_court_ratio,
108     }
109
110     # If a certain amount of court color is visible, consider it relevant
111     if avg_green_ratio < min_court_ratio:
112         return ValidationResult(
113             validator_name=self.name,
114             passed=False,
115             message=f"Video rejected: Relevance check failed. The scene does not
contain a recognizable {sport_name} court (court area found: {round(avg_green_ratio * 100, 2)}%
vs min required {round(min_court_ratio * 100, 2)}%).",
116             details=details,
117         )
118
119     return ValidationResult(
120         validator_name=self.name,
121         passed=True,
122         message=f"Video identified as relevant to {sport_name} ({sport_name} court
structures found).",
123         details=details,
124     )
125

```

14. user (2026-07-08T05:45:43.245Z)

```

1     import json
2     import logging
3     import subprocess
4     from typing import Any
5
6     from backend.pipeline.validators.base import ValidationResult, VideoValidator
7     from backend.utils.ffmpeg import ffprobe_bin, ffprobe_timeout_sec, timeout_label
8
9     logger = logging.getLogger(__name__)
10
11
12     def _probe_meta(video_path: str) -> dict | None:
13         """Re-probe ffprobe metadata as a fallback when format_check hasn't run first."""
14         try:
15             timeout = ffprobe_timeout_sec()
16             result = subprocess.run(
17                 [
18                     ffprobe_bin(),
19                     "-v",
20                     "error",
21                     "-select_streams",
22                     "v:0",
23                     "-show_format",
24                     "-show_streams",
25                     "-of",
26                     "json",
27                     video_path,
28                 ],
29                 capture_output=True,
30                 text=True,
31                 check=True,
32                 timeout=timeout,
33             )
34             return json.loads(result.stdout)
35         except subprocess.TimeoutExpired:
36             logger.warning(
37                 "resolution_fps_check: fallback ffprobe timed out after %s",
38                 timeout_label(ffprobe_timeout_sec()),

```

```

39         )
40         return None
41     except (subprocess.CalledProcessError, json.JSONDecodeError, OSError):
42         return None
43
44
45     class ResolutionFPSValidator(VideoValidator):
46         @property
47         def name(self) -> str:
48             return "resolution_fps_check"
49
50         @property
51         def description(self) -> str:
52             return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
53
54         def validate(
55             self, video_path: str, config: Any, context: dict[str, Any]
56         ) -> ValidationResult:
57             meta = context.get("ffprobe_meta")
58             if not meta or not meta.get("streams"):
59                 # Fallback: re-probe instead of hard-failing (format_check may not
60                 # have run, e.g. if validator ordering changed).
61                 logger.warning(
62                     "resolution_fps_check: ffprobe_meta not in shared context ? "
63                     "re-probing. Ensure format_check runs before resolution_fps_check "
64                     "in the validator registry."
65                 )
66                 meta = _probe_meta(video_path)
67             if not meta or not meta.get("streams"):
68                 return ValidationResult(
69                     validator_name=self.name,
70                     passed=False,
71                     message="Video metadata is missing. Run format_check validator first.",
72                 )
73
74             stream = meta["streams"][0]
75             width = int(stream.get("width", 0))
76             height = int(stream.get("height", 0))
77
78             # Parse frame rate (e.g. "30/1" or "30000/1001")
79             fps_str = stream.get("r_frame_rate", "0/1")
80             try:
81                 num, den = map(int, fps_str.split("/"))
82                 fps = num / den if den != 0 else 0
83             except ValueError:
84                 fps = 0.0
85
86             details = {"width": width, "height": height, "fps": round(fps, 2)}
87
88             # Check bounds from typed config (AppConfig.validation)
89             val_cfg = config.validation
90             min_w = val_cfg.min_resolution_width
91             min_h = val_cfg.min_resolution_height
92             min_fps = val_cfg.min_fps
93
94             if width < min_w or height < min_h:
95                 return ValidationResult(
96                     validator_name=self.name,
97                     passed=False,
98                     message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
99                     details=details,
100                 )
101

```

```

102         if fps < min_fps:
103             return ValidationResult(
104                 validator_name=self.name,
105                 passed=False,
106                 message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
107                 details=details,
108             )
109
110         return ValidationResult(
111             validator_name=self.name,
112             passed=True,
113             message="Video resolution and frame rate meet minimum standards.",
114             details=details,
115         )
116

```

15. user (2026-07-08T05:45:43.595Z)

```

1     import subprocess
2     from typing import Any
3
4     from backend.pipeline.validators.base import ValidationResult, VideoValidator
5     from backend.utils.ffmpeg import ( # P2a: single ffmpeg/ffprobe resolver
6         ffprobe_bin,
7         ffprobe_timeout_sec,
8         timeout_label,
9     )
10
11
12     class PTSContinuityValidator(VideoValidator):
13         @property
14         def name(self) -> str:
15             return "pts_continuity_check"
16
17         @property
18         def description(self) -> str:
19             return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
timeline gaps, jumps, or editing spllices."
20
21         def validate(
22             self, video_path: str, config: Any, context: dict[str, Any]
23         ) -> ValidationResult:
24             # Command to extract packet PTS and flags (extremely fast, works on H.264 &
H.265)
25             cmd = [
26                 ffprobe_bin(),
27                 "-v",
28                 "error",
29                 "-select_streams",
30                 "v:0",
31                 "-show_entries",
32                 "packet=pts_time,flags",
33                 "-of",
34                 "csv=p=0",
35                 video_path,
36             ]
37
38             try:
39                 timeout = ffprobe_timeout_sec()
40                 result = subprocess.run(
41                     cmd, capture_output=True, text=True, check=True, timeout=timeout
42                 )
43                 output = result.stdout.strip()
44

```

```

45         if not output:
46             return ValidationResult(
47                 validator_name=self.name,
48                 passed=False,
49                 message="No packet timestamps could be extracted from the video
stream.",
50             )
51
52         timestamps = []
53         for line in output.split("\n"):
54             line = line.strip()
55             if line:
56                 parts = line.split(",")
57                 if len(parts) == 2:
58                     pts_str, flags = parts
59                     if "K" in flags: # Check if keyframe
60                         try:
61                             timestamps.append(float(pts_str))
62                         except ValueError:
63                             continue
64
65         if len(timestamps) < 2:
66             return ValidationResult(
67                 validator_name=self.name,
68                 passed=True,
69                 message="Video is too short to perform a PTS continuity check.",
70                 details={"keyframes_count": len(timestamps)},
71             )
72
73         # Analyze gaps
74         max_gap = 0.0
75         backward_jumps = 0
76         gaps = []
77
78         for i in range(1, len(timestamps)):
79             gap = timestamps[i] - timestamps[i - 1]
80             gaps.append(gap)
81             if gap < 0:
82                 backward_jumps += 1
83             if gap > max_gap:
84                 max_gap = gap
85
86         # A continuous GoPro recording usually has keyframe intervals (GOP size)
87         # of 1.0 to 4.0 seconds. A gap larger than the configured threshold
88         # (or a backward jump) indicates edited/spliced content.
89         details = {
90             "keyframes_count": len(timestamps),
91             "max_keyframe_gap_seconds": round(max_gap, 2),
92             "backward_jumps_detected": backward_jumps,
93             "first_keyframe": timestamps[0],
94             "last_keyframe": timestamps[-1],
95         }
96
97         if backward_jumps > 0:
98             return ValidationResult(
99                 validator_name=self.name,
100                 passed=False,
101                 message="Video integrity compromise: Backward timeline jumps
(splicing) detected.",
102                 details=details,
103             )
104
105         max_gap_threshold = config.validation.pts_max_gap_seconds
106         if max_gap > max_gap_threshold:
107             return ValidationResult(

```

```

108         validator_name=self.name,
109         passed=False,
110         message=f"Video integrity compromise: Detected a timestamp gap of
{round(max_gap, 2)}s, indicating edited or missing content.",
111         details=details,
112     )
113
114     return ValidationResult(
115         validator_name=self.name,
116         passed=True,
117         message="PTS timeline is continuous with no abnormal gaps or backward
jumps.",
118         details=details,
119     )
120
121     except subprocess.CalledProcessError as e:
122         return ValidationResult(
123             validator_name=self.name,
124             passed=False,
125             message="Failed to execute ffprobe PTS analysis.",
126             details={"error": e.stderr},
127         )
128     except subprocess.TimeoutExpired:
129         return ValidationResult(
130             validator_name=self.name,
131             passed=False,
132             message=f"ffprobe PTS analysis timed out after
{timeout_label(ffprobe_timeout_sec())}.",
133         )
134     except Exception as e:
135         return ValidationResult(
136             validator_name=self.name,
137             passed=False,
138             message=f"Unexpected error in PTS validation: {str(e)}",
139         )
140

```

16. user (2026-07-08T05:45:49.590Z)

```

180     )
181
182     def backup_database(self, dest: str) -> str:
183         """Write a consistent copy of the DB to ``dest`` (an 'export my data'
affordance, P2d).
184         Checkpoints the WAL first so ``dest`` is self-contained. Returns ``dest``."""
185         parent = os.path.dirname(os.path.abspath(dest))
186         if parent:
187             os.makedirs(parent, exist_ok=True)
188         with self._connect() as conn:
189             conn.execute("PRAGMA wal_checkpoint(FULL)")
190             shutil.copy2(self.db_path, dest)
191         return dest
192
193     def _create_base_tables(self, cursor):
194         """Create the videos / play_segments / events base tables (idempotent)."""
195         # Videos Table
196         cursor.execute("""
197             CREATE TABLE IF NOT EXISTS videos (
198                 id TEXT PRIMARY KEY,
199                 filepath TEXT NOT NULL,
200                 processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
201                 status TEXT NOT NULL,
202                 validation_results TEXT
203             )
204         """)

```

```

205
206 # Play Segments Table (Extensible metadata JSON)
207 cursor.execute("""
208     CREATE TABLE IF NOT EXISTS play_segments (
209         id INTEGER PRIMARY KEY AUTOINCREMENT,
210         video_id TEXT NOT NULL,
211         start_time REAL NOT NULL,
212         end_time REAL NOT NULL,
213         duration REAL NOT NULL,
214         server TEXT,
215         receiver TEXT,
216         metadata TEXT,
217         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
218     )
219 """)
220
221 # Events Table
222 cursor.execute("""
223     CREATE TABLE IF NOT EXISTS events (
224         id INTEGER PRIMARY KEY AUTOINCREMENT,
225         segment_id INTEGER NOT NULL,
226         timestamp REAL NOT NULL,
227         type TEXT NOT NULL,
228         player TEXT,
229         details TEXT,
230         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
231     )
232 """)
233
234 def _migrate_to_v1(self, cursor):
235     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
236     # Common fields are columns; detector-specific metrics go in `stats` JSON,
237     # so a new segmenter reuses this table by setting its own `detector`/`stats`.
238     cursor.execute("""
239         CREATE TABLE IF NOT EXISTS candidate_segments (
240             id INTEGER PRIMARY KEY AUTOINCREMENT,
241             video_id TEXT NOT NULL,
242             detector TEXT NOT NULL,
243             chunk_index INTEGER,
244             start_time REAL NOT NULL,
245             end_time REAL NOT NULL,
246             duration REAL NOT NULL,
247             confidence REAL,
248             stats TEXT,
249             detection_params TEXT,
250             confirmed_segment_id INTEGER,
251             human_label TEXT,
252             notes TEXT,
253             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
254             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
255             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id) ON
DELETE SET NULL
256         )
257         """)
258     cursor.execute("""
259         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
260             ON candidate_segments(video_id, detector)
261         """)
262     cursor.execute("PRAGMA user_version = 1")
263
264 def _migrate_to_v2(self, cursor):
265     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per submitted
266     # /api/process request. `status` is the EXECUTION state of the job
267     # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
268     # that failed validation) lives inside `result` JSON as result["status"].

```

```

269         # `result` is the full sync-era response dict (incl. report paths), so the
270         # observability report is the job's artifact, per the plan.
271         cursor.execute("""
272             CREATE TABLE IF NOT EXISTS jobs (
273                 id TEXT PRIMARY KEY,
274                 video_path TEXT NOT NULL,
275                 video_id TEXT,
276                 segmenter TEXT,
277                 player_pool TEXT,
278                 max_frames INTEGER,
279                 status TEXT NOT NULL DEFAULT 'queued',
280                 progress TEXT,
281                 error TEXT,
282                 result TEXT,
283                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
284                 started_at TIMESTAMP,
285                 finished_at TIMESTAMP
286             )
287         """)
288         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON jobs(status)")
289         cursor.execute("PRAGMA user_version = 2")
290
291     def _migrate_to_v3(self, cursor):
292         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2). `policy` = the
293         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting' job is
294         # due
295         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ? any worker
296         # re-claims it via `claim_due_job` when due ? with NO master node: the table IS
297         # the coordinator. ALTER (not recreate) so existing v2 job rows survive; the
298         # ADD COLUMNS are applied idempotently so an interrupted upgrade can re-run
299         # them.
300         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN policy TEXT")
301         self._add_column_idempotent(

```

17. user (2026-07-08T05:45:50.794Z)

```

455         # submit?worker boundary ? the worker reconstructs the ProcessRequest from this
456         # row, so a
457         # NULL/unset value (CLI / older clients / pre-v8 rows) means "use the server
458         # default", i.e.
459         # byte-identical to today. Additive-only ALTER; idempotent so an interrupted
460         # upgrade re-runs.
461         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN compute TEXT")
462         cursor.execute("PRAGMA user_version = 8")
463
464     def _migrate_to_v9(self, cursor):
465         # v9: the jobs table now holds TWO kinds of work ? `kind`='process' (the
466         # original, NULL ?
467         # process for pre-v9 rows) and `kind`='compile' (the async reel stitch, #329;
468         # /api/compile
469         # is now 202+poll like /api/process so a slow ffmpeg stitch can't trip the edge
470         # 524). `params`
471         # is the kind-specific JSON payload a compile job carries
472         # (segment_ids/output_filename/locale)
473         # since those don't map onto the process columns. Additive-only ALTERs;
474         # idempotent.
475         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN kind TEXT")
476         self._add_column_idempotent(cursor, "ALTER TABLE jobs ADD COLUMN params TEXT")
477         cursor.execute("PRAGMA user_version = 9")
478
479     def _migrate_to_v10(self, cursor):
480         # v10: validation-result CACHE keyed by the content id. A video's id IS the MD5
481         # of its bytes
482         # (backend/utils/hashing.py::compute_video_id), so a REUPLOAD of identical
483         # content keys the

```

```

474         # same row ? and when that content was ALREADY validated under the SAME
validator config, the
475         # pipeline reuses the stored verdict instead of re-decoding the clip through the
whole validator
476         # suite (minutes on a heavy 4K clip on the CPU-only control-plane; the
2026-07-07 CUJ re-paid it
477         # on every retry after a segmentation failure). Columns:
478         #   fingerprint ? ValidationRegistry.config_fingerprint (the validation.* config
+ sport +
479         #           validator set + a schema version); a lookup reuses the verdict
ONLY on a
480         #           fingerprint match, so a config/sport/validator change misses ?
re-validates.
481         #   passed      ? the overall verdict (1/0), so a cached FAIL short-circuits
without a decode.
482         #   results     ? the JSON ValidationResult list, replayed into the response +
report.
483         # Standalone table (NO foreign key): the row is written right after validation
runs ? which can
484         # be BEFORE the videos UPSERT exists ? and is dropped explicitly by
clear_video_data. UPSERT on
485         # the video_id PK keeps ONE (latest) verdict per video. Additive-only ? nothing
pre-existing
486         # changes; an install that never reuploads behaves byte-identically to v9.
487         cursor.execute("""
488             CREATE TABLE IF NOT EXISTS validation_cache (
489                 video_id TEXT PRIMARY KEY,
490                 fingerprint TEXT NOT NULL,
491                 passed INTEGER NOT NULL,
492                 results TEXT NOT NULL,
493                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
494             )
495         """)
496         cursor.execute("PRAGMA user_version = 10")
497
498
499         # Domain CRUD methods live in mixins so Database remains the public facade while
500         # schema/migration/connection logic stays centralized in this module.
501

```

18. user (2026-07-08T05:45:57.928Z)

```

11-
12-     def _execute_write( # pragma: no cover - implemented by Database facade
13-         self, sql: str, params: Tuple[Any, ...], error_msg: str
14-     ) -> bool:
15-         raise NotImplementedError
16-
17-     def add_video(
18-         self,
19-         video_id: str,
20-         filepath: str,
21-         status: str,
22-         validation_results: List[Dict[str, Any]],
23-     ) -> bool:
24-         """Register or update a video row WITHOUT touching its child segments.
25-
26-         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
27-         with foreign_keys ON ? cascade-deletes every play_segment/event/candidate_segment.
28-         That made a status update (e.g. marking 'failed' on re-validation, or 'processed'
29-         before the segmenter runs) silently destroy prior good results. UPSERT mutates the
30-         row in place; clearing segments is now an explicit, deliberate operation
31-         (clear_video_data / prune_segments).
32-         """
33-         return self._execute_write(

```



```

34:         "INSERT INTO videos (id, filepath, status, validation_results) VALUES (?, ?, ?,
35: ?) "
36:         "ON CONFLICT(id) DO UPDATE SET "
37:         "filepath=excluded.filepath, status=excluded.status, "
38:         "validation_results=excluded.validation_results",
39:         (video_id, filepath, status, json.dumps(validation_results)),
40:         "Failed to add video",
41:     )
42:     def set_validation_cache(
43:         self,
44:         video_id: str,
45:         fingerprint: str,
46:         passed: bool,
47:         validation_results: List[Dict[str, Any]],
48:     ) -> bool:
49:         """Cache a video's validation verdict keyed by its content id (the MD5 ``video_id``),
so an
50:         identical-bytes REUPLOAD can skip re-running the (minutes-long on heavy 4K) validator
suite.
51:         ``fingerprint`` is ``ValidationRegistry.config_fingerprint(config)`` ? a hash of the
52:         validation config + sport + validator set + a schema version. A reader
53:         (``get_validation_cache``)
54:         only reuses the verdict while that fingerprint still matches the CURRENT config, so a
config
55:         change re-validates instead of serving a stale verdict. ``passed`` is the overall
verdict and
56:         ``validation_results`` the full ValidationResult list (dumped) so a cached FAIL can
replay the
57:         same message and a cached PASS carries the same results into the report/response.
UPSERT on the
58:         ``video_id`` PK keeps ONE (latest) verdict per video: a re-validation under a NEW
config
59:         overwrites the stale row rather than accumulating history."""
60:         return self._execute_write(
61:             "INSERT INTO validation_cache (video_id, fingerprint, passed, results) "
62:             "VALUES (?, ?, ?, ?) "
63:             "ON CONFLICT(video_id) DO UPDATE SET "
64:             "fingerprint=excluded.fingerprint, passed=excluded.passed, "
65:             "results=excluded.results, created_at=CURRENT_TIMESTAMP",
66:             (video_id, fingerprint, 1 if passed else 0, json.dumps(validation_results)),
67:             "Failed to write validation cache",
68:         )
69:
70:     def get_validation_cache(self, video_id: str) -> Optional[Dict[str, Any]]:
71:         """The cached validation verdict for ``video_id`` ? ``{fingerprint, passed,
results}`` ? or
72:         ``None`` when nothing is cached.
73:
74:         The caller compares ``fingerprint`` against the CURRENT
75:         ``ValidationRegistry.config_fingerprint(config)`` and reuses the verdict ONLY on a
match (a
76:         config / sport / validator-set change flips the fingerprint ? re-validate).
``results`` is the
--
78:         segmentation and a cached FAIL returns the same failure ? in both cases without re-
decoding the
79:         video."""
80:         with self._connect() as conn:
81:             row = (
82:                 conn.cursor()
83:                 .execute(
84:                     "SELECT fingerprint, passed, results FROM validation_cache "
85:                     "WHERE video_id = ?",

```

```

86-         (video_id,),
87-     )
88-     .fetchone()
89- )
90- if not row:
91-     --
268-     d = dict(row)
269-     d["stats"] = json.loads(d["stats"] or "{}")
270-     d["detection_params"] = json.loads(d["detection_params"] or "{}")
271-     candidates.append(d)
272- return candidates
273-
274- def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
275-     with self._connect() as conn:
276-         row = (
277-             conn.cursor()
278-             .execute("SELECT * FROM videos WHERE id = ?", (video_id,))
279-             .fetchone()
280-         )
281-         if row:
282-             d = dict(row)
283-             d["validation_results"] = json.loads(d["validation_results"] or "[]")
284-             return d
285-         return None
286-
287- def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
288-     """All indexed videos, most-recently-added first. Orders by rowid (a monotonic
insertion
289-     counter, since id is a TEXT PK) rather than the 1-second-granularity processed_at,
which
290-     would tie within a batch. Mirrors get_video's row shape (validation_results parsed).
Powers
291-     GET /api/videos for the operator console (PACKAGING_PLAN.md P1)."""
292-     sql = "SELECT * FROM videos ORDER BY rowid DESC"
293-     params: tuple = ()
294-     if limit is not None:
295-         sql += " LIMIT ?"
296-         params = (int(limit),)
297-     with self._connect() as conn:
298-         rows = conn.cursor().execute(sql, params).fetchall()
299-     out: List[Dict[str, Any]] = []
300-     for row in rows:
301-         d = dict(row)
302-         d["validation_results"] = json.loads(d["validation_results"] or "[]")
303-         out.append(d)
304-     return out
305-
306- def query_play_segments(
307-     self, video_id: str, filters: Dict[str, Any]
308- ) -> List[Dict[str, Any]]:
309-     --
358-     return segments_list
359-
360- def clear_video_data(self, video_id: str):
361-     with self._connect() as conn:
362-         cur = conn.cursor()
363-         cur.execute("DELETE FROM videos WHERE id = ?", (video_id,))
364-         # Drop the cached validation verdict too ? the validation_cache row has no FK to
videos
365-         # (it can be written before the videos row exists), so an explicit clear must
remove it
366-         # here or the next upload would reuse a verdict for data the caller just wiped.
367-         cur.execute("DELETE FROM validation_cache WHERE video_id = ?", (video_id,))
368-         conn.commit()
369-

```

19. user (2026-07-08T05:46:05.051Z)

```
1      """Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.
2
3      python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
4          [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
5      python -m backend.worker train ...          # Gen-0 train-from-bucket (follow-up)
6
7      The URI scheme selects the storage backend via ``backend.storage`` (a bare path /
8      ``local://``
9      is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing
10     local runs
11     are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
12     ``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini
13     key) ?
14     the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).
15
16     This is a thin orchestrator: it composes the SAME building blocks the ``backend.main``
17     CLI uses
18     (validator registry ? segmenter ? report), never a forked pipeline.
19     """
20
21     from __future__ import annotations
22
23     import argparse
24     import csv
25     import json
26     import logging
27     import os
28     import re
29     import sys
30     import tempfile
31     from typing import Any, List
32
33     from backend import storage
34     from backend.compute import ComputeJobSpec, get_compute_backend
35     from backend.config import (
36         StartupCheckError,
37         enforce_startup_checks,
38         load_config,
39         probe_cuda_available,
40     )
41     from backend.infrastructure.database import Database
42     from backend.pipeline.segmenters.base import segmenter_registry
43     from backend.pipeline.validators.base import registry as validator_registry
44     from backend.reporting import emit_indexer_report
45     from backend.results import Failure
46     from backend.utils.ffmpeg import ffprobe_bin # P2a: single ffmpeg/ffprobe resolver
47     from backend.utils.hashing import compute_video_id
48
49     logger = logging.getLogger("backend.worker")
50
51     def _coerce(v: str) -> Any:
52         """Coerce a ``--set`` string value to bool/int/float where it round-trips, else
53         str."""
54         low = v.lower()
55         if low in ("true", "false"):
56             return low == "true"
57         for cast in (int, float):
58             try:
59                 return cast(v)
60             except ValueError:
61                 pass
62         return v
```

```

59
60
61     def _apply_overrides(config: Any, sets: List[str]) -> None:
62         """Apply ``a.b.c=value`` overrides onto the typed config (e.g.
indexing.skip_ai_handoff=true)."""
63         for kv in sets or []:
64             key, sep, val = kv.partition("=")
65             if not sep:
66                 raise ValueError(f"--set expects k=v, got {kv!r}")
67             parts = key.split(".")
68             obj = config
69             for p in parts[:-1]:
70                 obj = getattr(obj, p)
71                 setattr(obj, parts[-1], _coerce(val))
72
73
74     def _write_intervals_csv(segments: List[dict], path: str) -> None:
75         """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-
friendly
76         artifact (same schema as the rally-annotator labels)."""
77         with open(path, "w", newline="") as f:
78             w = csv.writer(f)
79             w.writerow(["start", "end", "shot_count", "ending_reason"])
80             for s in segments:
81                 w.writerow(
82                     [
83                         f"{float(s.get('start_time', 0.0)):.3f}",
84                         f"{float(s.get('end_time', 0.0)):.3f}",
85                         s.get("shot_count", ""),
86                         s.get("ending_reason", s.get("shot_count_confidence", "")),
87                     ]
88                 )
89
90
91     def _write_report_json(
92         video_id: str, segments: List[dict], status: str, path: str, video_uri: str = ""
93     ) -> None:
94         with open(path, "w") as f:
95             json.dump(
96                 {
97                     "video_id": video_id,
98                     "status": status,
99                     "segment_count": len(segments),
100                     # The SOURCE the detection ran on (CP-rebuild P3, #485): a restart-wiped
101                     # control plane re-hydrating from outputs/<md5>/ restores the video row
102                     # with its truthful path ? without this, only the segments recover.
103                     "video_uri": video_uri,
104                     "segments": [
105                         {
106                             "start": float(s.get("start_time", 0.0)),
107                             "end": float(s.get("end_time", 0.0)),
108                         }
109                         for s in segments
110                     ],
111                 },
112                 f,
113                 indent=2,
114             )
115
116
117     def _serialize_and_push_outputs(
118         scratch: str,
119         out_uri: str,
120         video_id: str,
121         segments: List[dict],

```

```

122         status: str,
123         storage_cfg: dict,
124         video_uri: str = "",
125     ) -> List[str]:
126         """Write intervals.csv + report.json under ``<scratch>/outputs/<video_id>/`` and
push both to
127         ``<out_uri>/outputs/<video_id>/``; return the pushed URIs.
128
129         Shared by the SUCCESS path and the failure path (``_fail``) so a detector
timeout/abort leaves a
130         ``status="failed"`` report.json in the bucket ? the control plane's source of truth
131         (``orchestration.cached_process_result`` / ``probe_process_cache`` treat any
non-``success``
132         report as "no usable result" ? retry, and the restart-recovery poll waits for
report.json to
133         EXIST) ? instead of a missing artifact or a false 0-segment ``"success"``. """
134         out_local = os.path.join(scratch, "outputs", video_id)
135         os.makedirs(out_local, exist_ok=True)
136         _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
137         _write_report_json(
138             video_id,
139             segments,
140             status,
141             os.path.join(out_local, "report.json"),
142             video_uri=video_uri,
143         )
144         out_prefix = out_uri.rstrip("/")
145         pushed: List[str] = []
146         for fn in sorted(os.listdir(out_local)):
147             uri = f"{out_prefix}/outputs/{video_id}/{fn}"
148             storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
149             pushed.append(uri)
150         return pushed
151
152
153     def _run_startup_checks(config: Any) -> bool:
154         """M5 per-profile startup checks for the worker (the compute process, so it probes
CUDA
155         best-effort). Logs warnings; returns False on a FATAL coherence error so the caller
aborts
156         cleanly (rc 2) rather than run an expensive job under an incoherent profile. A true
no-op
157         (returns True, ZERO work) when no deployment.profile is selected.
158
159         The CUDA probe is gated behind an active profile: ``probe_cuda_available()`` lazily
imports
160         torch, and pre-M5 the worker's infer/train path was torch-free (detection is
delegated to a
161         WSL/native subprocess). Probing only when a profile is set keeps the default-OFF
path a true
162         no-op of work, not just of output. """
163         profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
164         cuda = (
165             probe_cuda_available() if profile else None
166         ) # torch-free on the default-OFF path
167         try:
168             enforce_startup_checks(config, cuda_available=cuda, logger=logger)
169             return True
170         except StartupCheckError:
171             return False # already logged at ERROR by enforce_startup_checks
172
173
174     def _dispatch_remote(compute: Any, args: argparse.Namespace) -> int:
175         """M6: hand ONE infer job to a remote ComputeBackend (Cloud Run) and return its rc
once the

```

```

176         outputs are durably in the bucket (the backend bucket-polls the done-marker). The
dispatched
177         container runs this SAME cmd_infer INLINE (compute_target forced local_gpu) ? never
re-dispatch.
178
179         A remote job that never writes a completion marker exits via one of two CLEAN codes
(distinct
180         from the local 2/3), never a raw traceback:
181         * rc 4 ? the bucket poll exhausted its budget (`PolicyTimeout`): the execution
may still be
182         running (the shim best-effort cancels it).
183         * rc 5 ? the execution has TERMINATED without a marker
(`RemoteExecutionFailed`): a worker
184         crash/OOM/pre-marker abort caught FAST by the Executions-API probe, so instead
of the
185         ~65-min opaque wait the operator gets an immediate, log-pointing failure.""
186         from backend.compute import RemoteExecutionFailed
187         from backend.infrastructure.execution_policy import PolicyTimeout
188
189         spec = ComputeJobSpec(
190             video_uri=args.video_uri,
191             out_uri=args.out_uri,
192             segmenter=args.segmenter,
193             config_overrides=tuple(args.set or ()),
194             skip_validation=bool(getattr(args, "skip_validation", False)),
195         )
196         try:
197             result = compute.run_infer(spec)
198         except RemoteExecutionFailed as e:
199             logger.error(
200                 "[worker] remote execution terminated without a completion marker: %s", e
201             )
202             return 5
203         except PolicyTimeout as e:
204             logger.warning(
205                 "[worker] remote compute did not complete (no marker within budget): %s", e
206             )
207             return 4
208         logger.info(
209             "[worker] remote compute done: video_id=%s rc=%d outputs=%s",
210             result.video_id,
211             result.returncode,
212             result.outputs_uri,
213         )
214         return result.returncode
215
216
217     def _write_done_marker(
218         done_uri: str,
219         video_id: str,
220         returncode: int,
221         segment_count: int,
222         status: str,
223         storage_cfg: dict,
224         scratch: str,
225     ) -> None:
226         """M6: write a tiny JSON completion marker to ``done_uri`` (via the storage layer)
for a remote
227         dispatcher's bucket-poll. Best-effort ? never raises into the worker's success
return.""
228         try:
229             marker = {
230                 "video_id": video_id,
231                 "returncode": returncode,
232                 "segment_count": segment_count,

```

```

233         "status": status,
234     }
235     local = os.path.join(scratch, "_done.json")
236     with open(local, "w", encoding="utf-8") as f:
237         json.dump(marker, f)
238         storage.put(local, done_uri, config=storage_cfg)
239         logger.info("[worker] wrote completion marker -> %s", done_uri)
240 except Exception as e: # never fail a good run because the marker push hiccuped
241     logger.warning("[worker] could not write done-marker %s: %s", done_uri, e)
242
243
244 def cmd_infer(args: argparse.Namespace) -> int:
245     config = load_config(args.config) if args.config else load_config()
246     _apply_overrides(config, args.set)
247     if not _run_startup_checks(config):
248         return 2
249
250     # M6: a cloud compute_target DISPATCHES to a Cloud Run Job instead of running in-
process.
251     # DEFAULT-OFF: get_compute_backend returns None for ""/local_gpu/cpu_mock ? fall
through to the
252     # unchanged inline path below. The dispatched container runs THIS cmd_infer inline
(the
253     # dispatcher forces compute_target=local_gpu), so it never recursively re-
dispatches.
254     compute = get_compute_backend(config)
255     if compute is not None:
256         return _dispatch_remote(compute, args)
257
258     storage_cfg = config.storage.model_dump()
259
260     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
261     os.makedirs(scratch, exist_ok=True)
262     config.output.output_dir = scratch
263
264     # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download
to scratch.
265     local_video = storage.fetch(
266         args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg
267     )
268     print(f"[worker] pulled {args.video_uri} -> {local_video}")
269
270     # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
byte-identical).
271     video_id = compute_video_id(local_video)
272
273     # 3. VALIDATE (same registry as the main CLI) ? UNLESS the dispatcher already did.
274     # On the cloud-serving path the CONTROL-PLANE is the authoritative validation gate:
it runs the
275     # SAME validators with ITS resolved config and only dispatches on PASS. Re-
validating here is
276     # redundant, decodes the whole video a SECOND time, and is outright WRONG when the
worker's config
277     # resolves different thresholds than the control-plane's ? observed 2026-07-07: a
5312x2988 clip
278     # (sharpness 11.9) PASSED the control-plane at validation.min_blur_threshold=8.0,
then the worker
279     # REJECTED it at its own default 20.0, so a SUCCESSFUL GPU dispatch failed after the
fact.
280     # --skip-validation (set by the dispatcher in orchestration._maybe_dispatch_remote)
skips it; the
281     # direct ``worker infer`` CLI has no control-plane, so it still validates by
default.
282     if getattr(args, "skip_validation", False):
283         print(

```

```

284         "[worker] validation SKIPPED (--skip-validation): the dispatcher is the
authoritative "
285         "gate and already validated this input."
286     )
287     val_results, val_context = [], {}
288 else:
289     val_results, val_context = validator_registry.run_all_with_context(
290         local_video, config
291     )
292     failures = [r for r in val_results if not r.passed]
293     db = Database(os.path.join(scratch, config.output.db_name))
294     db.add_video(
295         video_id,
296         local_video,
297         "failed" if failures else "processed",
298         [r.model_dump() for r in val_results],
299     )
300
301     def _fail(rc: int) -> int:
302         # A worker-side FAILURE (validation, unknown segmenter, or a detector
timeout/abort the
303         # segmenter surfaced as a Failure) must NOT masquerade as a 0-rally "success".
Write + push
304         # a status="failed" report.json/intervals.csv (0 segments) so the control
plane's bucket
305         # truth (submit-cache probe, restart-recovery) and the alpha-user result surface
see an
306         # explicit FAILURE ? the wf294 silent-failure fix. Best-effort: a push hiccup
must never
307         # mask the original failure. THEN the M6 done-marker (real rc + status) so a
remote
308         # dispatcher's bucket-poll terminates FAST instead of hanging until its budget
expires.
309         try:
310             _serialize_and_push_outputs(
311                 scratch,
312                 args.out_uri,
313                 video_id,
314                 [],
315                 "failed",
316                 storage_cfg,
317                 str(getattr(args, "video_uri", "")) or "",
318             )
319         except Exception as e: # never mask the real failure on an artifact-push hiccup
320             logger.warning("[worker] could not push failure artifacts: %s", e)
321         # DEFAULT-OFF: no --done-uri ? no marker written (the unchanged local run).
322         if getattr(args, "done_uri", None):
323             _write_done_marker(
324                 args.done_uri, video_id, rc, 0, "failed", storage_cfg, scratch
325             )
326         return rc
327
328     segments: List[dict] = []
329     segmenter_actual = None
330     segmentation_ran = False
331     status = "failed"
332     try:
333         if failures:
334             print(
335                 "[worker] validation FAILED: "
336                 + "; ".join(f"{r.validator_name}: {r.message}" for r in failures)
337             )
338             return _fail(2)
339     seg_name = args.segmenter or config.indexing.default_segmenter
340     segmenter_cls = segmenter_registry.get(seg_name)

```



```

341         if not segmenter_cls:
342             print(
343                 f"[worker] unknown segmenter: {seg_name!r} "
344                 f"(have {list(segmenter_registry.list_available())})"
345             )
346             return _fail(2)
347         segmenter_actual = seg_name
348         result = segmenter_cls(db, config).process_video(
349             video_id, local_video, max_frames=args.max_frames
350         )
351         segmentation_ran = True
352         if isinstance(result, Failure):
353             print(f"[worker] segmenter FAILED: {result.message}")
354             return _fail(3)
355         segments = result.value if hasattr(result, "value") else result
356         status = "success"
357         # Loud, never-silent diagnostic: a 0-segment success is almost always a
misconfig, not an
358         # empty match ? the cold-start failure mode (the substrate detector yielded no
usable
359         # trajectories). Explain it so a run is analysable from the log alone (re-run -v
for more).
360         if not segments:
361             detector_impl = (
362                 os.environ.get("RALLY_DETECTOR_IMPL")
363                 or getattr(config.indexing, "detector_impl", None)
364                 or "auto"
365             )
366             logger.warning(
367                 "infer produced 0 segments for video_id=%s (segmenter=%s,
detector_impl=%s). "
368                 "The detector substrate yielded no usable trajectories/windows ? check
the detector "
369                 "logs above; common causes: a missing ${AI_PROVIDER}_API_KEY with "
370                 "skip_ai_handoff=False (the segmenter bails BEFORE detection ? set the
key or "
371                 "indexing.skip_ai_handoff=true; cloud-serving now fails this at
startup), native "
372                 "runner UNAVAILABLE (weights/repo/WASB_PYTHON env), the WASB env
produced no "
373                 "detections, or the input resolution differs from the tuned proxy
resolution. "
374                 "Re-run with -v for the native command + frame counts.",
375                 video_id,
376                 segmenter_actual,
377                 detector_impl,
378             )
379         finally:
380             # Best-effort per-video observability report (never raises) ? parity with the
main CLI.
381             emit_indexer_report(
382                 db=db,
383                 video_id=video_id,
384                 video_path=local_video,
385                 config=config,
386                 val_results=val_results,
387                 val_context=val_context,
388                 segmenter_requested=args.segmenter,
389                 segmenter_actual=segmenter_actual,
390                 was_reingest=False,
391                 segmentation_ran=segmentation_ran,
392             )
393
394         # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
395         pushed = _serialize_and_push_outputs(

```

```

396         scratch,
397         args.out_uri,
398         video_id,
399         segments,
400         status,
401         storage_cfg,
402         str(getattr(args, "video_uri", "") or ""),
403     )
404
405     print(
406         f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}"
407     )
408     for u in pushed:
409         print("    ", u)
410
411     # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
412     # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
413     # write their own marker via _fail() above, so the dispatcher fails FAST either way
414     # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
415     # if getattr(args, "done_uri", None):
416     #     _write_done_marker(
417     #         args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418     #     )
419     return 0
420
421
422     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
423     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
424     _VIDEO_EXTS = ("mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
425
426     #: Leading ISO-date prefix on a canonical label name (`labels/<YYYY-MM-
427     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
428     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
429     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
430
431
432     def _label_clip_name(fname: str) -> str:
433         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
434         filename.
435
436         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
437         prefix, so a
438         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
439
440         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094`` (canonical, date-
441         stamped)
442         * ``GX020094.rallies.csv`` -> ``GX020094`` (legacy bare ? same
443         stem)
444         * ``GX020094_proxy.rallies.csv`` -> ``GX020094_proxy`` (``_proxy`` is part of
445         the name, kept)
446         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
447         <name> survive)
448
449         Without the date strip, ``--trajectory-source detector`` can't match a dated label
450         to its bare
451         video stem, so every clip is skipped and train aborts with ``<2 videos``

```

```

(DATA_IN_GCS $7b #1).
445     """
446     name = fname.replace(".rallies.csv", "").replace(".csv", "")
447     return _LABEL_DATE_PREFIX.sub("", name)
448
449
450 def _probe_video_fps_width(path: str):
451     """Robust ffmpeg (fps, frame_width) for a real video. Returns None on ANY failure.
452
453     The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
454     fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
455     cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
456     inference report (it flags the fallback) but corrupting for REAL training data. So
here we
457     probe with ffmpeg (same tool as ``get_video_duration``) and return None so the
caller SKIPS
458     rather than guesses.
459     """
460     import json
461     import subprocess
462
463     try:
464         out = subprocess.check_output(
465             [
466                 ffmpeg_bin(),
467                 "-v",
468                 "error",
469                 "-select_streams",
470                 "v:0",
471                 "-show_entries",
472                 "stream=r_frame_rate,width",
473                 "-of",
474                 "json",
475                 path,
476             ],
477             stderr=subprocess.DEVNULL,
478             ).decode()
479         st = (json.loads(out).get("streams") or [{}])[0]
480         num, _, den = str(st.get("r_frame_rate", "0/1")).partition("/")
481         fps = float(num) / float(den or "1")
482         width = float(st.get("width") or 0)
483         if fps > 0 and width > 0:
484             return fps, width
485     except (
486         subprocess.SubprocessError,
487         ValueError,
488         KeyError,
489         OSError,
490         json.JSONDecodeError,
491     ):
492         pass
493     return None
494
495
496 def _resolve_train_detector(args: argparse.Namespace, config: Any):
497     """Build the trajectory DetectorRunner for ``--trajectory-source detector``, honouring
498     detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU
box,
499     stub=CPU mock). Returns the runner, or prints an error + returns None.
500
501     Reuses the segmenter's ``_resolve_runner`` seam so the worker and the live CLI build
the

```

```

502 detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo)
has
503 no shuttle detector and is rejected.
504 """
505 from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
506
507 seg_cls = segmenter_registry.get(args.segmenter)
508 if not seg_cls:
509     print(
510         f"[worker] unknown segmenter: {args.segmenter!r} "
511         f"(have {list(segmenter_registry.list_available())})"
512     )
513     return None
514 seg = seg_cls(None, config)
515 if not isinstance(seg, TrajectoryHybridSegmenter):
516     print(
517         f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
518         f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid). "
519     )
520     return None
521 try:
522     runner, _ = seg._resolve_runner(config.indexing)
523 except NotImplementedError as e:
524     # e.g. detector_impl='native' for a family without a native runner (tracknet, or
525     # fusion with a tracknet substrate). Fail cleanly (rc!=0), not with a raw
526     traceback.print_exc()
527     print(f"[worker] detector not available: {e}")
528     return None
529 ok, msg = runner.healthcheck()
530 if not ok:
531     print(f"[worker] detector environment not ready: {msg}")
532     return None
533 print(f"[worker] detector ready ({args.segmenter}): {msg}")
534 return runner
535
536 def cmd_train(args: argparse.Namespace) -> int:
537     """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest ->
538     shell out
539     to training.gen0.harness (backend must NOT import training) -> push the artifact
540     bundle.
541     Two trajectory sources (the train pipeline + harness are identical either way):
542     * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent
543     synthetic
544     shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box /
545     CI.
546     * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
547     DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test).
548     This
549     is the M3.5 "first real training" path; only the trajectory source flips.
550     """
551     import subprocess
552
553     config = load_config(args.config) if args.config else load_config()
554     _apply_overrides(config, args.set)
555     if not _run_startup_checks(config):
556         return 2
557     storage_cfg = config.storage.model_dump()
558
559     scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
560     labels_dir = os.path.join(scratch, "labels")
561     traj_dir = os.path.join(scratch, "trajectories")
562     videos_dir = os.path.join(scratch, "videos")
563     os.makedirs(labels_dir, exist_ok=True)

```

```

560     os.makedirs(traj_dir, exist_ok=True)
561
562     corpus = args.corpus_uri.rstrip("/")
563     label_uris = sorted(
564         u
565         for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
566         if u.lower().endswith(".csv")
567     )
568     if len(label_uris) < 2:
569         print(
570             f"[worker] need >=2 labeled videos for leave-one-video-out; found
{len(label_uris)} "
571             f"under {corpus}/labels/"
572         )
573         return 2
574
575     # Real-detector source: resolve the runner once + index the bucket's videos/ by
stem.
576     runner = None
577     video_by_stem: dict = {}
578     if args.trajectory_source == "detector":
579         os.makedirs(videos_dir, exist_ok=True)
580         runner = _resolve_train_detector(args, config)
581         if runner is None:
582             return 2
583         for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
584             vname = storage.parse_uri(vuri).name
585             if not vname.lower().endswith(_VIDEO_EXTS):
586                 continue # skip sidecars (.srt/.json/thumbnails) so they can't shadow
the video
587             video_by_stem[os.path.splitext(vname)[0]] = vuri
588
589     print(f"[worker] trajectory source: {args.trajectory_source}")
590     specs: List[dict] = []
591     for uri in label_uris:
592         fname = storage.parse_uri(uri).name # e.g. 2026-06-22_GX020094.rallies.csv
593         name = _label_clip_name(fname) # -> GX020094 (date prefix stripped)
594         local_label = storage.fetch(
595             uri, os.path.join(labels_dir, fname), config=storage_cfg
596         )
597         if args.trajectory_source == "detector":
598             vuri = video_by_stem.get(name)
599             if not vuri:
600                 print(
601                     f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping."
602                 )
603                 continue
604             local_video = storage.fetch(
605                 vuri,
606                 os.path.join(videos_dir, storage.parse_uri(vuri).name),
607                 config=storage_cfg,
608             )
609             video_id = compute_video_id(local_video)
610             # Real fps/width drive the trajectory frame->time mapping; PROBE (don't
guess) ? a
611             # wrong fps silently mis-aligns every label, so skip the video if ffprobe
can't read it.
612             meta = _probe_video_fps_width(local_video)
613             if meta is None:
614                 print(
615                     f"[worker] could not probe fps/width for {name!r} (ffprobe) ?
skipping (won't guess)."
616                 )
617                 continue
618             fps, frame_width = meta

```

```

619         traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
620     if not traj:
621         print(
622             f"[worker] detector produced no trajectory for {name!r} ? skipping."
623         )
624         continue
625     specs.append(
626         {
627             "name": name,
628             "trajectory": traj,
629             "labels": local_label,
630             "fps": fps,
631             "frame_width": frame_width,
632         }
633     )
634 else:
635     from backend.pipeline.detectors.stub_runner import (
636         synth_trajectory_from_labels,
637     )
638
639     traj = os.path.join(traj_dir, f"{name}_traj.csv")
640     synth_trajectory_from_labels(
641         local_label, traj, fps=args.fps, frame_width=args.frame_width
642     )
643     specs.append(
644         {
645             "name": name,
646             "trajectory": traj,
647             "labels": local_label,
648             "fps": args.fps,
649             "frame_width": args.frame_width,
650         }
651     )
652
653     if len(specs) < 2:
654         print(
655             f"[worker] need >=2 videos with trajectories for leave-one-video-out; got
{len(specs)}."
656         )
657         return 2
658     manifest_path = os.path.join(scratch, "manifest.json")
659     with open(manifest_path, "w", encoding="utf-8") as f:
660         json.dump(specs, f, indent=2)
661     src_label = (
662         "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
663     )
664     print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")
665
666     out_dir = os.path.join(scratch, "artifacts")
667     cmd = [
668         sys.executable,
669         "-m",
670         "training.gen0.harness",
671         "--manifest",
672         manifest_path,
673         "--out",
674         out_dir,
675         "--version",
676         args.version,
677     ]
678     if args.floor:
679         floor_local = (
680             storage.fetch(
681                 args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg
682             )

```

```

683         if "://" in args.floor
684         else args.floor
685     )
686     cmd += ["--floor", floor_local]
687     print("[worker] training:", " ".join(cmd))
688     rc = subprocess.run(cmd).returncode
689     if rc != 0:
690         print(f"[worker] gen0 harness failed (rc={rc})")
691         return rc
692
693     # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
694     bundle = os.path.join(out_dir, args.version)
695     out_prefix = args.out_uri.rstrip("/")
696     pushed = []
697     for fn in sorted(os.listdir(bundle)):
698         uri = f"{out_prefix}/weights/{args.version}/{fn}"
699         storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
700         pushed.append(uri)
701     print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
702     for u in pushed:
703         print("    ", u)
704     return 0
705
706
707 def build_parser() -> argparse.ArgumentParser:
708     p = argparse.ArgumentParser(
709         prog="python -m backend.worker",
710         description="Storage-aware worker: pull ? run pipeline ? push.",
711     )
712     p.add_argument(
713         "-v",
714         "--verbose",
715         action="store_true",
716         help="DEBUG logging (the native detector command, frame counts, per-stage
detail)",
717     )
718     sub = p.add_subparsers(dest="cmd", required=True)
719
720     pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
721     pi.add_argument(
722         "--video-uri", required=True, help="local path / local:// / s3:// / gcs://"
723     )
724     pi.add_argument(
725         "--out-uri",
726         required=True,
727         help="output prefix (outputs/<video_id>/? is appended)",
728     )
729     pi.add_argument(
730         "--segmenter", default=None, help="default: indexing.default_segmenter"
731     )
732     pi.add_argument(
733         "--config", default=None, help="config.json path (default: ./config.json)"
734     )
735     pi.add_argument(
736         "--scratch", default=None, help="local scratch dir (default: a temp dir)"
737     )
738     pi.add_argument("--max-frames", type=int, default=None)
739     pi.add_argument(
740         "--done-uri",
741         default=None,
742         help="M6: storage URI for a completion MARKER (video_id + rc) written on a "
743         "successful run, so a remote (Cloud Run) dispatcher's bucket-poll terminates. "
744         "Default-OFF: omit it for the normal local run.",
745     )
746     pi.add_argument(

```

```

747         "--set",
748         action="append",
749         default=[],
750         metavar="k=v",
751         help="config override, e.g. --set indexing.skip_ai_handoff=true",
752     )
753     pi.add_argument(
754         "--skip-validation",
755         action="store_true",
756         help="skip the worker's re-validation: the dispatcher (control-plane) already
ran the "
757         "validators with its resolved config and only dispatches on PASS, so re-
validating here is "
758         "redundant, decodes the video again, and can contradict the gate. Default-OFF:
the direct "
759         "`worker infer` CLI still validates.",
760     )
761     pi.set_defaults(func=cmd_infer)
762
763     pt = sub.add_parser(
764         "train",
765         help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)",
766     )
767     pt.add_argument(
768         "--corpus-uri",
769         required=True,
770         help="prefix containing labels/ (+ videos/ for "
771         "--trajectory-source detector), e.g. s3://bucket or gcs://bucket",
772     )
773     pt.add_argument(
774         "--out-uri",
775         required=True,
776         help="output prefix (weights/<version>/ is appended)",
777     )
778     pt.add_argument("--version", required=True, help="semver for the artifact bundle")
779     pt.add_argument(
780         "--trajectory-source",
781         choices=["synth", "detector"],
782         default="synth",
783         help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
784         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
785         "(needs videos/ in the corpus; honours detector_impl).",
786     )
787     pt.add_argument(
788         "--segmenter",
789         default="wasb_hybrid",
790         help="trajectory hybrid whose detector is used for --trajectory-source
detector",
791     )
792     pt.add_argument(
793         "--floor", default=None, help="floor baseline JSON (local path or storage URI)"
794     )
795     pt.add_argument("--config", default=None)
796     pt.add_argument("--scratch", default=None)
797     pt.add_argument(
798         "--fps",
799         type=float,
800         default=30.0,
801         help="synth trajectory fps (detector probes the video)",
802     )
803     pt.add_argument(
804         "--frame-width",
805         type=float,
806         default=1280.0,
807         help="synth trajectory frame width (detector probes the video)",

```



```

808     )
809     pt.add_argument("--set", action="append", default=[], metavar="k=v")
810     pt.set_defaults(func=cmd_train)
811     return p
812
813
814     def _setup_logging(verbose: bool) -> None:
815         """Configure logging for the worker CLI so the pipeline's existing INFO/ERROR
816         diagnostics are
817         VISIBLE ? the worker previously ran with no logging config, so every
818         segmenter/detector log
819         (detector choice, 'Running native WASB inference', availability failures) was
820         silently dropped.
821         That is exactly why a 0-rally cold-start gave nothing to analyse. Configured here
822         (the CLI
823         entry), not at import, so importing the module never reconfigures a caller's
824         logging."""
825         # force=True: basicConfig is a no-op if the root already has handlers (e.g. a
826         library
827         # configured logging at import), which would leave the level at WARNING and re-
828         suppress the
829         # diagnostics. Forcing reconfiguration means the worker reliably controls its own
830         verbosity.
831         logging.basicConfig(
832             level=logging.DEBUG if verbose else logging.INFO,
833             format="%(asctime)s [%(levelname)s] %(name)s: %(message)s",
834             datefmt="%H:%M:%S",
835             force=True,
836         )
837
838
839     def main(argv=None) -> int:
840         args = build_parser().parse_args(argv)
841         _setup_logging(getattr(args, "verbose", False))
842         return args.func(args)
843
844
845     if __name__ == "__main__":
846         sys.exit(main())
847
848

```

20. user (2026-07-08T05:46:18.486Z)

```

27-     running): this says the execution is DEFINITELY over and no marker will ever appear, so
28-     the
29-     dispatcher must stop bucket-polling immediately instead of burning the full
30-     ``remote_poll_max_wait_sec`` (~65 min) on a guaranteed-futile wait. The message carries
31-     the
32-     job + execution name + terminal state + the ``gcloud`` command to read the execution log,
33-     so
34-     the whole "worker crashed before the done-marker" class fails fast AND diagnostically."""
35-
36-
37- @dataclass(frozen=True)
38- class ComputeJobSpec:
39-     """One inference job, profile-agnostic. ``video_id`` is the MD5 the worker will
40-     (re)compute
41-     from the SAME input bytes, so the output always lands at
42-     ``<out_uri>/outputs/<video_id>/`` ?
43-     deterministic regardless of where the job ran."""
44-
45-     video_uri: str # input (gs://? for cloud; a path/local:// for local)
46-     out_uri: str # output ROOT (gs://? bucket for cloud)
47-     segmenter: Optional[str] = None
48-     # Raw worker ``--set k=v`` overrides. A tuple (not list/dict) so the spec stays

```

```

frozen/hashable.
44-     config_overrides: tuple[str, ...] = ()
45-     # Skip the worker's redundant re-validation. The DISPATCHER (the cloud-serving control-
plane) is
46-     # the authoritative validation gate: it runs the SAME validators with ITS resolved config
and only
47-     # dispatches on PASS, so a worker re-run is (a) redundant, (b) a second full-video
decode, and
48-     # (c) outright WRONG when the worker's config resolves different thresholds ? it can
REJECT a clip
49-     # the gate already passed. Default False keeps the direct ``worker infer`` CLI (no
control-plane)
50-     # validating exactly as before.
51-     skip_validation: bool = False
52-
53-
54-@dataclass(frozen=True)
55-class ComputeResult:
56-     """The terminal state of a dispatched job. ``returncode`` mirrors the worker's exit code
57-     (0 ok / 2 validation-or-config / 3 segmenter-failure), plus **4 = remote dispatch timeout
or an
58-     empty/unreadable completion marker** (the cloud path only). ``video_id`` is known once
the job's
59-     done-marker is read; ``outputs_uri`` is where ``intervals.csv`` / ``report.json``
landed."""
60-
61-     returncode: int
62-     outputs_uri: str = ""
63-     video_id: str = ""
64-     report: dict = field(default_factory=dict)
65-     # The backend's execution identity for PROVENANCE (e.g. the Cloud Run execution name) ?
surfaced
66-     # on the job result so "the cloud path actually ran" is independently verifiable (gcloud
run jobs
67-     # executions describe <execution>). Empty for the in-process path / backends without one.
68-     execution: str = ""
69-
70-
71-class ComputeBackend(ABC):

```

21. user (2026-07-08T05:46:19.720Z)

```

256-         spec.video_uri,
257-         "--out-uri",
258-         spec.out_uri,
259-         "--done-uri",
260-         done_uri,
261-     ]
262-     if spec.segmenter:
263-         argv += ["--segmenter", spec.segmenter]
264-     if spec.skip_validation:
265-         # The control-plane already validated + only dispatches on PASS ? tell the
worker not to
266-         # re-decode and re-validate (redundant, and its config may resolve different
thresholds).
267-         argv.append("--skip-validation")
268-     for kv in (*spec.config_overrides, *self._container_overrides):
269-         argv += ["--set", kv]
270-
271-     execution = self._client.run_job(
272-         self._job_name, argv, region=self._region, project=self._project
273-     )
274-     LOG.info(
275-         "cloud-run: dispatched job=%s exec=%s; bucket-polling %s",

```

22. user (2026-07-08T05:46:29.653Z)

```
15-Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16-
17-
18-class ValidationRelevanceConfig(BaseModel):
19:    court_green_lower: list[int] = Field(default=[35, 40, 40])
20:    court_green_upper: list[int] = Field(default=[85, 255, 255])
21:    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22-
23-
24:class ValidationConfig(BaseModel):
25:    min_resolution_width: int = 1280
26:    min_resolution_height: int = 720
27:    min_fps: float = 29.9
28:    min_blur_threshold: float = 20.0
29:    max_scene_cuts_per_minute: float = 0.5
30:    pts_max_gap_seconds: float = Field(
31:        default=10.0,
32:        ge=0.0,
33:        description=(
34:            "Maximum allowed keyframe gap (seconds) before flagging as spliced. "
```

23. user (2026-07-08T05:46:33.419Z)

```
11: while an incremental MD5 accumulates ? the assembled file never touches local
15: ``input_root``) plus the streamed md5 as ``video_id``.
27:import hashlib
43:from backend.utils.hashing import compute_video_id # canonical file-identity hashing
185:    "hasher": hashlib.md5(),
254:    st["hasher"].update(data)
288:    """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
300:    # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301:    # part IS compute_video_id's digest (same bytes, same algorithm), so /api/process
316:    "video_id": st["hasher"].hexdigest(),
329:    video_id = compute_video_id(final_path)
332:    "video_id": video_id,
```

24. user (2026-07-08T05:46:44.450Z)

```
5:that raises `JobWaiting` is parked (status 'waiting' + `next_poll_at` + progress) and
9:every cross-function seam ? `db_instance`, `global_config`, `_execute_processing`,
109: is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm contract.
137:    # v8 compute-picker: the worker runs _execute_processing ? _maybe_dispatch_remote,
which
146:    """Run ONE job already claimed (`status='running`) by `claim_due_job`, recording its
149:    Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done' means
151:    result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means the
163:    result = _main._execute_compile(
171:    result = _main._execute_processing(
177:    if result.get("video_id"):
178:        db.set_job_video_id(job_id, result["video_id"])
179:        db.mark_job_done(job_id, result)
182:        job, result, config, db
185:        job, result, config
197:    job: Dict[str, Any], result: Dict[str, Any], config: Any, db: Any
200:    ``config.billing.enabled``. When on, compute the job's cost from ``result['usage']``
204:    ? **Honest gap (M8 scope):** the pipeline does **not** emit ``result['usage']`` yet, so
even with
211:    claimed row (carries ``owner_id``); ``result`` is the pipeline output."""
217:    usage = (result or {}).get("usage") or {}
223:    compute_backend=(result or {}).get("compute_backend"),
239:    job: Dict[str, Any], result: Dict[str, Any], config: Any
248:    row (carries ``owner_id``); ``result`` is the pipeline output."""
```

```

256:         flywheel.run_for_job(job, result, config)
265:     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is picked
413:         "control plane restarted during a forced reprocess; the cached result "
457:         "control plane restarted mid-run and the cloud worker's result never "
458:         "appeared within the poll budget; re-submit the video (a finished result "
462:     payload = orchestration.cached_process_result(
468:         "[JOBS] resumed job %s: ingested the in-flight worker's result for %s.",
473:     # The report exists but did not realize into a usable result ? surface its verdict.
474:     status = "unreadable"
477:     status = str(report.get("status") or "unreadable")
482:     f"cloud worker finished without a usable result after a control-plane restart "
483:     f"(report status: {status}); re-submit the video",

```

25. user (2026-07-08T05:46:55.866Z)

```

280         "upload_id": upload_id,
281         "received_parts": st["parts"],
282         "received_bytes": st["bytes"],
283     }
284
285
286     @router.post("/api/upload/{upload_id}/complete")
287     def upload_complete(upload_id: str, req: UploadCompleteRequest):
288         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
289         upload_dir, _, _, _ = _upload_cfg()
290         with _uploads_lock:
291             st = _uploads.get(upload_id)
292             if st is None:
293                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
294             if st["parts"] == 0 or st["parts"] != req.total_parts:
295                 raise HTTPException(
296                     status_code=409,
297                     detail=f"Expected {req.total_parts} part(s), received {st['parts']}.",
298                 )
299             if st.get("writer") is not None:
300                 # Remote staging: close() finalizes the streamed object; the md5 accumulated per
301                 # part IS compute_video_id's digest (same bytes, same algorithm), so
/api/process
302                 # gets the identity without anyone re-reading the video. (Sync endpoint ?
FastAPI
303                 # runs it on the threadpool, so the blocking finalize doesn't stall the loop.)
304                 try:
305                     st["writer"].close()
306                 except Exception as e: # noqa: BLE001 ? surface a clean verdict, then clean up
307                     _abort_upload(upload_id)
308                     raise HTTPException(
309                         status_code=502,
310                         detail=f"Failed to finalize the staged upload ({e}). Re-init to retry.",
311                     ) from e
312                 with _uploads_lock:
313                     _uploads.pop(upload_id, None)
314                 return {
315                     "path": st["stage_uri"],
316                     "video_id": st["hasher"].hexdigest(),
317                     "size_bytes": st["bytes"],
318                     "next": "POST /api/process with this path",
319                 }
320             final_path = os.path.join(upload_dir, st["filename"])
321             if os.path.exists(
322                 final_path
323             ): # never overwrite an earlier upload with the same name
324                 stem, ext = os.path.splitext(st["filename"])
325                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
326             os.replace(st["tmp_path"], final_path)

```

```

327         with _uploads_lock:
328             _uploads.pop(upload_id, None)
329         video_id = compute_video_id(final_path)
330         return {
331             "path": os.path.abspath(final_path),
332             "video_id": video_id,
333             "size_bytes": os.path.getsize(final_path),
334             "next": "POST /api/process with this path",
335         }
336
337
338     @router.delete("/api/upload/{upload_id}")
339     def upload_abort(upload_id: str):

```

26. user (2026-07-08T05:47:07.548Z)

```

59:     enforce_startup_checks,
94: from backend.utils.hashing import compute_video_id # canonical file-identity hashing
111: ``request.app.state`` ? eliminating the Optional module-level globals that forced
258: class ProcessRequest(BaseModel):
266:     # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267:     # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268:     # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269:     compute: Optional[str] = None
272:     force: bool = False
277:     # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is written
verbatim.
278:     video_id: str
289:     # the whole-file MD5 (compute_video_id, D3) so it joins the pipeline by content hash.
297:     video_id: str
305:     video_id: str
366:     """Honest probe of what THIS box can do ? the first-run wizard renders the compute step
from
401:         "compute_target": getattr(deployment, "compute_target", "") or "",
402:         # item 3: can this server satisfy a per-request compute="cloud" dispatch? The
SPA shows
439:     "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
448:         "video_id": row.get("id"),
461:     (``video_id`` + a friendly ``match_name`` derived from the filename) so the library
shows real
482:     does NOT track a video_id?reel association today (a per-video listing would be a
phantom), so
512:     """Stream a compiled reel by its bare filename (no video_id ? reels are flat in
output_dir).
549: def _source_cache_lock(video_id: str) -> threading.Lock:
551:     return _source_cache_locks.setdefault(video_id, threading.Lock())
554: def _source_cache_path(video_id: str, input_ref, cfg) -> str:
559:     name = safe_output_filename(f"{video_id}{ext}")
563: def _cached_remote_source(video_id: str, input_ref, cfg) -> str:
565:     cache_path = _source_cache_path(video_id, input_ref, cfg)
568:     lock = _source_cache_lock(video_id)
591: def _warm_annotation_proxy(video_id: str, src_local: str, cfg) -> None:
596:     if video_id in _proxy_inflight:
598:         _proxy_inflight.add(video_id)
605:         src_local, os.path.join(_proxies_dir(cfg), f"{video_id}.mp4")
608:         logger.warning("proxy: warm failed for %s: %s", video_id, e)
611:         _proxy_inflight.discard(video_id)
614: @app.get("/api/source/{video_id}")
615: def get_source(video_id: str, request: Request):
621:     rec = request.app.state.db.get_video(video_id)
628:     proxy_name = safe_output_filename(f"{video_id}.mp4")

```

```

655:         local = _cached_remote_source(video_id, input_ref, cfg)
667:         target=_warm_annotation_proxy, args=(video_id, local, cfg), daemon=True
677:         ``<output_dir>/labels/<video_id>.rallies.csv`` and keyed by the file-MD5 ``video_id`` ?
so it joins
681:     Edge-auth mirrors /api/process (default-OFF ? no-op locally). ``video_id`` is validated
as a bare
691:         name = safe_output_filename(f"{req.video_id}.rallies.csv")
693:         raise HTTPException(status_code=400, detail=f"invalid video_id: {e}") from e
707:         "annotations saved: video_id=%s rows=%d -> %s", req.video_id, num_rows, path
720: def _asset_dest_uri(medium_uri: str, video_id: str, ext: str) -> str:
721:     """The durable, MD5-keyed destination inside the chosen medium:
``<medium>/<video_id><ext>``. """
722:     return f"{medium_uri.rstrip('/')}/{video_id}{ext}"
727:     enforced, D11), or ``None`` for a cloud bucket (which is cost/quota-warned, never
blocked).
753:     MD5 (``compute_video_id``, D3) so it joins the pipeline by content hash, and appears in
822:         video_id = compute_video_id(local_working)
841:         # "This computer" ? the upload already lives here; it IS the durable copy
(no duplicate).
844:         dest = _asset_dest_uri(req.medium_uri, video_id, ext)
865:         "[ASSETS] put failed video_id=%s dest=%s: %s",
866:         video_id,
876:         if not request.app.state.db.add_video(video_id, stored_uri, "stored", []):
879:             "[ASSETS] stored video_id=%s medium=%s uri=%s copied=%s owner=%s",
880:             video_id,
889:             "video_id": video_id,
962:         # bucket (no drain, no GPU dispatch). `force` opts out. Logic in backend.api.jobs.
992: @app.get("/api/videos/{video_id}/cost")
993: def get_video_cost(video_id: str, request: Request):
996:     agg = request.app.state.db.get_video_cost(video_id)
1015:         "video_id": job.get("video_id"),
1041: @app.get("/api/highlights/{video_id}/{filename}")
1042: def download_highlight(video_id: str, filename: str, request: Request):
1050:     if not request.app.state.db.get_video(video_id):
1057:         signed = signed_reel_url(request.app.state.config, video_id, name)
1061:         logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)

```

[Showing results with pagination = limit: 70]

27. user (2026-07-08T05:47:14.960Z)

```

255
256
257     # --- API Models ---
258     class ProcessRequest(BaseModel):
259         video_path: str
260         player_pool: Optional[List[str]] = None
261         max_frames: Optional[int] = None
262         segmenter: Optional[str] = None
263         # The UI locale the SPA was rendered in (e.g. "kn", "hi-IN"). Recorded on the job
for PMF
264         # analytics (count_jobs_by_locale). Optional ? NULL when unrecorded (CLI / older
clients).
265         locale: Optional[str] = None
266         # SPA compute-picker (item 3): "local" forces in-process even on a cloud-configured
server;
267         # "cloud" forces a Cloud Run dispatch (requires deployment.cloud_available). None ?
server
268         # default (deployment.compute_target). Only honoured for these two values; see
_maybe_dispatch_remote.
269         compute: Optional[str] = None
270         # Explicit opt-in reprocess path. By default, a video whose content hash already has
stored
271         # play_segments is returned from cache so refresh/Try again cannot re-bill paid

```

```

segmenters.
272         force: bool = False
273
274
275     class AnnotationsRequest(BaseModel):
276         # Finished rally labels for a video, as the annotator's byte-compatible CSV (P8,
from the Studio
277         # AnnotatePanel). `video_id` is the file MD5 (the pipeline's join key); `csv` is
written verbatim.
278         video_id: str
279         csv: str

```

28. user (2026-07-08T05:47:16.045Z)

```

1090         if not root or "://" not in root:
1091             return ""
1092         return f"{root.rstrip('/')}/highlights/{video_id}/{name}"
1093
1094
1095     def _storage_settings(cfg) -> Optional[Dict[str, Any]]:
1096         """The active storage settings dict (per-backend config for the storage layer ? S3
endpoint/region,
1097         GCS project/client/signed_url_ttl), or None. Threaded into storage calls so a non-
default backend
1098         isn't silently constructed as a fresh default."""
1099         sc = getattr(cfg, "storage", None)
1100         return sc.model_dump() if sc is not None else None
1101
1102
1103     def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1104         """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1105         307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1106         /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1107         the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1108         reel_uri = reel_object_uri(cfg, video_id, name)
1109         if not reel_uri:
1110             return None
1111         settings = _storage_settings(cfg)
1112         if storage.exists(reel_uri, config=settings):
1113             return storage.signed_url(reel_uri, config=settings, method="GET")
1114         return None
1115
1116
1117     def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1118         """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments,
stitch the reel,
1119         and return the {status, filepath, download_url} result the SPA polls for. Returns a
1120         ``status='failed'`` result for an EXPECTED problem (missing video / no segments / a
stitch error)
1121         so the SPA always gets a clean verdict instead of a worker-crash error."""
1122         import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
1123
1124         notify = progress or (lambda stage: None)
1125         params = json.loads(job.get("params") or "{}")
1126         video_id = job.get("video_id") or ""
1127         segment_ids = params.get("segment_ids") or []
1128         locale = params.get("locale")
1129
1130         notify("resolving")
1131         try:

```

```

1132         video, kept, out_name = _resolve_compile(
1133             db, cfg, video_id, segment_ids, params.get("output_filename", "")
1134         )
1135     except _CompileError as e:
1136         return {"status": "failed", "message": e.detail, "video_id": video_id}
1137
1138     stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1139     output_dir = (
1140         getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1141     )
1142     # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1143     # unknown/absent locale keeps the configured default unchanged.
1144     localized_watermark = localize_watermark(stitching.watermark, locale)
1145     compiler = _main.VideoCompiler(
1146         output_dir,
1147         begin_padding=stitching.begin_padding,
1148         end_padding=stitching.end_padding,
1149         watermark=localized_watermark,
1150     )
1151     time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1152
1153     notify("stitching")
1154     local_src = ""
1155     try:
1156         # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1157         # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1158         # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1159         local_src = storage.acquire_local_input(
1160             video["filepath"], getattr(cfg, "storage", None)
1161         )
1162         # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1163         # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1164         with _LOCAL_COMPUTE_LANE:
1165             out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1166     except Exception as e: # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1167         logger.error(
1168             "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1169         )
1170         return {
1171             "status": "failed",
1172             "message": f"Highlight stitching failed: {e}",
1173             "video_id": video_id,
1174         }
1175     finally:
1176         if local_src:
1177             storage.cleanup_acquired_local_input(
1178                 video["filepath"], local_src, getattr(cfg, "storage", None)
1179             )
1180
1181     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1182     # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1183     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1184     # still falls back to the local file on the same instance).
1185     reel_uri = reel_object_uri(cfg, video_id, out_name)
1186     if reel_uri:

```



```

1187         try:
1188             notify("uploading reel")
1189             # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1190             # project/client) is used, not a fresh default.
1191             storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1192         except Exception as e: # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1193             logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1194
1195         # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1196         # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1197         download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1198         notify("finished")
1199         return {

```

29. user (2026-07-08T05:47:27.506Z)

```

230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234             logger.info(
235                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",
236                 getattr(input_ref, "uri", input_ref),
237                 exc_info=True,
238             )
239             return None
240
241
242     def _cached_process_payload(
243         video_id: str, validation_results, segments, requested_compute
244     ) -> Dict[str, Any]:
245         """The /api/process result for a video whose detection already exists ? ONE shape
for
246         both producers (the drain's DB-cache short-circuit and the submit-time cache,
#484)."""
247         return {
248             "video_id": video_id,
249             "status": "success",
250             "message": (
251                 f"Reused {len(segments)} cached play segments. "
252                 "Pass force=true to reprocess this video."
253             ),
254             "results": validation_results or [],
255             "play_segments": segments,
256             "cached": True,
257             "compute": {"path": "cached", "requested": requested_compute},
258         }
259
260
261     def _read_bucket_json(uri: str, storage_config: dict) -> Optional[Dict[str, Any]]:
262         """Fetch + parse one small JSON object (cloud backends require a real dst ? the
263         ``_ingest_remote_segments`` pattern). Raises on fetch/parse errors; callers
decide."""
264         ref = storage.parse_uri(uri)
265         with tempfile.TemporaryDirectory(prefix="cr-report-") as td:
266             local = storage.fetch(
267                 uri, os.path.join(td, ref.name or "report.json"), config=storage_config
268             )
269             with open(local, encoding="utf-8") as f:
270                 loaded = json.load(f)
271             return loaded if isinstance(loaded, dict) else None
272

```

```

273
274 # One re-hydrate at a time per video_id (#492 review blocker 1): two concurrent cache
hits
275 # from an empty DB must not both ingest intervals.csv ? the loser waits, then finds the
276 # winner's rows in the DB and reuses them. Process-wide is enough: the CP is single-
instance
277 # (maxScale=1, spec D8) and SQLite serializes cross-process writers anyway.
278 _REHYDRATE_LOCKS: Dict[str, threading.Lock] = {}
279 _REHYDRATE_LOCKS_GUARD = threading.Lock()
280
281
282 def _rehydrate_lock(video_id: str) -> threading.Lock:
283     with _REHYDRATE_LOCKS_GUARD:
284         return _REHYDRATE_LOCKS.setdefault(video_id, threading.Lock())
285
286
287 def probe_process_cache(db, cfg, video_id: str) -> bool:
288     """READ-ONLY: does completed detection for ``video_id`` already exist (DB or
bucket)?
289
290     Safe to call before the atomic submit claim ? it writes nothing (#492 review blocker
1
291     ordering). Used to decide whether a submit can skip the local-scratch fetch guard
292     (blocker 2): a True probe means the request should never need CP scratch. Best-
effort:
293     errors report False (? the normal dispatch path with its guards)."""
294     try:
295         existing = db.get_video(video_id)
296         if (
297             existing
298             and existing.get("status") == "processed"
299             and db.query_play_segments(video_id, {})
300         ):
301             return True
302         sc = _storage_settings(cfg)
303         if not sc or (sc.get("backend") or "local") == "local":
304             return False
305         output_root = (sc.get("output_root") or "").rstrip("/")
306         if not output_root:
307             return False
308         report_uri = f"{output_root}/outputs/{video_id}/report.json"
309         if not storage.exists(report_uri, config=sc):
310             return False
311         report = _read_bucket_json(report_uri, sc)
312         return bool(report and report.get("status") == "success")
313     except Exception: # noqa: BLE001 ? a failed probe degrades to the guarded dispatch
path
314         logger.warning(
315             "[API] cache probe failed for %s ? treating as a miss", video_id,
exc_info=True
316         )
317     return False
318
319
320 def cached_process_result(
321     db, cfg, video_id: str, video_path: str, requested_compute=None
322 ) -> Optional[Dict[str, Any]]:
323     """A ready-to-serve /api/process result when this video's detection ALREADY exists ?
324     the DB first, then the bucket (``outputs/<md>report.json`` with ``status=success``
325     re-ingests via ``_ingest_remote_segments``): **the DB is a cache, the bucket is the
326     source of truth** (#484 instant re-runs; the #485 re-hydrate mechanism). ``None`` ?
no
327     completed work found (or the check failed) ? the caller dispatches normally. The
bucket
328     branch mirrors the real cloud ingest exactly (watermarks + ``_finalize_reingest``),

```

```

so a
329         partial earlier ingest is replaced, never double-counted.
330
331         Concurrency (#492 review blocker 1): the whole check-then-ingest runs under a
332         per-`video_id` lock ? a concurrent caller blocks, then finds the winner's rows via
the
333         in-lock DB re-check and reuses them (one segment set, ever). Callers must hold a
CLAIMED
334         job before invoking (writes happen post-claim)."""
335     try:
336         with _rehydrate_lock(video_id):
337             existing = db.get_video(video_id)
338             if existing and existing.get("status") == "processed":
339                 segs = db.query_play_segments(video_id, {})
340                 if segs:
341                     return _cached_process_payload(
342                         video_id,
343                         existing.get("validation_results", []),
344                         segs,
345                         requested_compute,
346                     )
347             sc = _storage_settings(cfg)
348             if not sc or (sc.get("backend") or "local") == "local":
349                 return None
350             output_root = (sc.get("output_root") or "").rstrip("/")
351             if not output_root:
352                 return None
353             outputs_uri = f"{output_root}/outputs/{video_id}"
354             report_uri = f"{outputs_uri}/report.json"
355             if not storage.exists(report_uri, config=sc):
356                 return None
357             report = _read_bucket_json(report_uri, sc)
358             if not report or report.get("status") != "success":
359                 return None

```

30. user (2026-07-08T05:47:37.270Z)

```

56: def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
108: def _ingest_remote_segments(
209: def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
423: def _maybe_dispatch_remote(

```

31. user (2026-07-08T05:47:38.865Z)

```

423:     output_root: str = ""
424:     # --- Input side (COMPUTE_DECOUPLED_SERVING M2; parallel to the output
`backend`/`output_root`).
427:     # input name is resolved against `input_root` (e.g.
`input_root="gs://bucket/videos"` + ".mp4"
428:     # => `gs://bucket/videos/x.mp4`) or, if `input_root` is empty, prefixed with
`input_backend`
433:     input_root: str = ""
440:     # Durable workspace root URI. "" => precedence: output_root, then output.output_dir.
460:     `deployment.profile` or the `DEPLOYMENT_PROFILE` env var); auto-detect only ever
471:     compute_target: Literal["", "local_gpu", "cloud_run", "cpu_mock"] = ""
473:     # bucket-poll CONCURRENTLY when compute_target=="cloud_run". Each is an I/O-bound Cloud
Run Job
549:     deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)

```

32. user (2026-07-08T05:47:44.701Z)

```

423     def _maybe_dispatch_remote(
424         cfg,
425         db,

```

```

426     req: "ProcessRequest",
427     video_id: str,
428     seg_name: str,
429     val_results,
430     play_wm: int,
431     cand_wm: int,
432     notify,
433 ) -> Optional[Tuple[Dict[str, Any], bool]]:
434     """Cloud-serving (COMPUTE_DECOUPLED_SERVING M6 ? the API path): when
435     ``deployment.compute_target
436     == "cloud_run"`` , run the heavy DETECT on a Cloud Run GPU Job and INGEST its rally
437     intervals into
438     THIS DB, so the rest of the flow (query ? compile/local-stitch) is unchanged.
439     Returns ``(response_dict, ran)`` when the cloud path is taken ? ``ran`` is whether
440     the cloud job
441     actually EXECUTED (``False`` for a pre-dispatch config rejection, so the
442     observability report
443     doesn't claim a fake segmentation run) ? or ``None`` to fall through to the in-
444     process segmenter
445     (**DEFAULT-OFF**: ``get_compute_backend`` returns ``None`` for every
446     non-``cloud_run`` target, so
447     the in-process path stays byte-identical)."""
448     from backend.compute import ComputeJobSpec, get_compute_backend
449
450     def _failed(msg: str, ran: bool) -> Tuple[Dict[str, Any], bool]:
451         # Shape-match the in-process segmenter-fail branch + drop any partial NEW rows
452         (id > play_wm);
453         # prior good rows (id <= play_wm) are preserved (destroy-AFTER-confirm).
454         db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
455         # Provenance even on failure, and HONEST about what ran: `path` is "cloud_run"
456         only when the
457         # job actually dispatched + executed remotely (ran=True, e.g. a non-zero worker
458         rc / ingest
459         # error); a pre-dispatch reject (ran=False) ran NOWHERE ? "not_run". `target`
460         records the
461         # intended backend, `requested` the picker choice, `dispatched` whether the
462         remote job ran.
463         return (
464             {
465                 "video_id": video_id,
466                 "status": "failed",
467                 "message": msg,
468                 "results": [r.model_dump() for r in val_results],
469                 "play_segments": [],
470                 "compute": {
471                     "path": "cloud_run" if ran else "not_run",
472                     "requested": req.compute,
473                     "target": "cloud_run",
474                     "dispatched": ran,
475                 },
476             },
477             ran,
478         )
479
480     # SPA compute-picker (item 3): a per-request choice overrides the server's
481     configured default.
482     # "local" forces the in-process segmenter; "cloud" forces a Cloud Run dispatch
483     (guarded by
484     # _cloud_available so we fail clearly rather than silently running local); None /
485     unknown ? default.
486     choice = (req.compute or "").strip().lower()
487     if choice and choice not in ("local", "cloud"):
488         logger.warning(
489             "[API] ignoring unknown compute=%r (expected 'local'/'cloud'); using server

```

```

default.",
477         req.compute,
478     )
479     choice = ""
480     if choice == "local":
481         # A hosted cloud-serving control-plane bakes NO detector weights into its image
(only the
482         # worker Cloud Run Job GCSFuse-mounts them), so forcing a weights-backed
trajectory detector
483         # to run IN-PROCESS here can never succeed ? it dies deep in the segmenter with
a cryptic
484         # "WASB weights not found" (observed live: job f29d41d0?, 2026-07-07). Fail FAST
with an
485         # actionable message instead of that silent trap. A truly-local box (not cloud-
capable), or a
486         # cloud-wired box that ACTUALLY has the weights on disk, is unaffected ? it runs
in-process
487         # below exactly as before (the weights-present check is the escape hatch).
488         if (
489             (seg_name or "").strip().lower() in _WEIGHTS_BACKED_SEGMENTERS
490             and _cloud_available(cfg)
491             and not _local_detector_weights_present(cfg, seg_name)
492         ):
493             return _failed(
494                 f"this hosted server can't run detection locally ? the '{seg_name}'
model weights "
495                 "live only on the cloud worker, not on the control-plane. Choose 'Run in
cloud'." ,
496                 False,
497             )
498             return None # explicit "run on my machine" ? in-process regardless of the
server's target
499         if choice == "cloud":
500             if not _cloud_available(cfg):
501                 return _failed(
502                     "cloud-serving was requested but this server isn't configured for it "
503                     "(no Cloud Run target). Pick 'run on my machine', or configure cloud
serving.",
504                     False,
505                 )
506             compute = get_compute_backend(cfg, target_override="cloud_run")
507         else:
508             compute = get_compute_backend(
509                 cfg
510             ) # server default (default-OFF unless cloud_run)
511         if compute is None:
512             return None # default-OFF ? run the in-process segmenter below (unchanged)
513
514         storage_cfg = getattr(cfg, "storage", None)
515
516         # The Cloud Run job fetches the input FROM THE BUCKET ? it can't read a path local
to this box.
517         try:
518             input_ref = storage.resolve_input_ref(req.video_path, storage_cfg)
519         except ValueError as e:
520             return _failed(
521                 f"cloud-serving: unresolvable input '{req.video_path}': {e}", False
522             )
523         if input_ref.backend == "local":
524             # A browser upload lands on THIS box's local disk; the Cloud Run job (a
different machine)
525             # can't read it. Stage it UP to the input bucket first, then dispatch from there
(#461). Only
526             # possible when a CLOUD input_root is configured; without one we genuinely can't
stage ? fail.

```

```

527         # Classify the ROOT ITSELF (parse_uri), not storage.input_backend: the staged
URI below is
528         # built by joining under input_root, and storage.put routes by
parse_uri(staged_uri).backend ?
529         # so an explicit cloud root (gs://?, s3://?) stages fine with input_backend left
at its
530         # default, while a bare/local-looking root can't stage no matter what
input_backend says
531         # (resolve_input_uri only consults input_backend when input_root is EMPTY, and
an empty root
532         # gives us nothing to build the staged URI under).
533         input_root = (getattr(storage_cfg, "input_root", "") or "").strip()
534         try:
535             root_backend = (
536                 storage.parse_uri(input_root).backend if input_root else "local"
537             )
538         except ValueError: # unsupported scheme in input_root ? nowhere real to stage
to
539             root_backend = "local"
540         if root_backend == "local":
541             return _failed(
542                 "cloud-serving needs a cloud input (e.g. gs://?), or a cloud
storage.input_root to "
543                 "stage local uploads into; a path local to this machine can't be read by
the Cloud "
544                 "Run job.",
545                 False,
546             )
547         local_src = input_ref.key # resolved local path of the upload
548         staged_uri =
f"{input_root.rstrip('/')}/{video_id}/{os.path.basename(local_src)}"
549         try:
550             notify("staging upload to bucket")
551             # Thread the active storage settings so backend-specific config survives (S3
endpoint_url/
552             # region/addressing for R2·D0·MinIO; GCS project/client) ? not a fresh
default client.
553             storage.put(
554                 local_src,
555                 staged_uri,
556                 config=(
557                     storage_cfg.model_dump() if storage_cfg is not None else None
558                 ),
559             )

```

33. user (2026-07-08T05:48:07.085Z)

```

30:     db: Any, job_id: str, req: Any, owner_id: Optional[str], video_id: str = ""
38:     hashed until the worker runs, so the submitted ref is the stable pre-flight key.
``video_id`` is
39:     the submit-time md5 when cheaply known (#484) ? persisted at CREATE so a restarted
control plane
51:         video_id=video_id,
73:     (hash + validate). Raises ``HTTPException(507)`` when the box can't hold the copy.""
86:     1. Resolve a cheap md5 (gs:// metadata) and PROBE the cache ? read-only, no side effects.
92:     per video_id) and completes the job instantly: no drain, no GPU dispatch.
98:     video_id = orchestration.submit_time_video_id(input_ref, cfg) or ""
99:     probed = bool(video_id) and not req.force and orchestration.probe_process_cache(
100:         db, cfg, video_id
104:     outcome = claim_process_job(db, job_id, req, owner_id, video_id=video_id)
113:         db, cfg, video_id, req.video_path, requested_compute=req.compute
185:         "video_id": j.get("video_id"),

```

34. user (2026-07-08T05:48:15.516Z)

```

196         are present so a forced per-request override can actually dispatch + collect a
result.
197
198         Surfaced in ``/api/capabilities`` so the SPA only offers the cloud toggle when it
would work. """
199         deployment = getattr(cfg, "deployment", None)
200         if (getattr(deployment, "compute_target", "") or "") == "cloud_run":
201             return True
202         has_job = bool(
203             os.environ.get("CLOUD_RUN_JOB") and os.environ.get("GOOGLE_CLOUD_PROJECT")
204         )
205         out_root = (getattr(getattr(cfg, "storage", None), "output_root", "") or "").strip()
206         return bool(has_job and out_root)
207
208
209     def _wait_progress_message(seg_name: str, elapsed_sec: float) -> str:
210         """The live job.progress line while a cloud GPU run is in flight (#482). Honest and
211         minute-granular: the control plane knows ELAPSED time, not frames ? frame-level %
comes
212         with the worker-side heartbeat (the #482 follow-up). Under a minute reads as
"starting"
213         (queue/cold-start), so the very first tick already shows movement. """
214         minutes = int(elapsed_sec) // 60
215         if minutes < 1:
216             return f"segmenting ({seg_name}) on cloud GPU ? starting up"
217         return f"segmenting ({seg_name}) on cloud GPU ? {minutes} min elapsed"
218
219
220     def submit_time_video_id(input_ref, cfg) -> Optional[str]:
221         """The video's md5 (the canonical ``video_id``) resolvable CHEAPLY at submit time ?
gs://
222         object metadata only, one stat, no bytes (#484). ``None`` ? unknown: local files
keep
223         deferred hashing on the drain (DEFERRED_HARDENING #16), and composite objects carry
no
224         ``md5Hash``. Never raises ? a metadata miss must not block a submit. """
225         if getattr(input_ref, "backend", "") != "gcs":
226             return None
227         sc = _storage_settings(cfg)
228         if not sc:
229             return None
230         try:
231             st = storage.get_storage("gcs", config=sc).stat(input_ref)
232             return getattr(st, "md5", None) or None
233         except Exception: # noqa: BLE001 ? best-effort identity, the drain still hashes
234             logger.info(
235                 "[API] submit-time md5 unavailable for %s (metadata stat failed)",

```

35. user (2026-07-08T05:48:24.809Z)

```

3:Pure-ish harvest: given the validation results/context, the persisted segments,
33:     val_context: Optional[Dict[str, Any]], video_path: Optional[str]
35:     """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
44:     ffmata = (val_context or {}).get("ffprobe_meta") or {}
83: def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
84:     with rec.stage("validation") as st:
85:         total = len(val_results)
86:         passed = sum(1 for r in val_results if getattr(r, "passed", False))
89:         failures = [r for r in val_results if not getattr(r, "passed", True)]
208:     val_results: List[Any],
209:     val_context: Optional[Dict[str, Any]],
231:     vm = video_meta_from_context(val_context, video_path)
243:     _validation_stage(rec, val_results)

```

36. assistant (2026-07-08T05:49:32.860Z)

I have a complete picture of the validation subsystem. Let me finalize the map.

37. user (2026-07-08T05:51:19.856Z)

Structured output provided successfully